VisionGuard

AI vs Real image classification, tracking and deployment
MLOps Assignment 2

Name	Abdul-Rehman
Roll number	23L-0787
Project name	23L-0787_A2_Ops
GitHub repository	https://github.com/Abdul-Rehman-0787/23L-0787_A2_Ops
DagsHub project	https://dagshub.com/Abdul-Rehman-0787/23L-0787_A2_Ops
Live model API (Render)	https://visionguard-vkzn.onrender.com (health check: /health)
Live web app (Render, bonus)	https://two3l-0787-a2-ops.onrender.com/
Graded version (commit)	Latest commit on branch main: https://github.com/Abdul-Rehman-0787/23L-0787_A2_Ops/commits/main

Disclosure of AI assistance

I used an AI assistant (Claude, made by Anthropic) throughout this project. It gave step-by-step guidance for each task, drafted the Python scripts, the Dockerfile, the Compose file, the GitHub Actions workflow, the tests, the cloud API and the web front end, explained concepts, and helped draft the wording of this report. I ran every command myself on my own machine and accounts, took every screenshot and checked the results against the logs. Where a suggestion did not work, I changed course; the clearest example is the cloud choice in Chapter 7, where Hugging Face Spaces turned out to need a paid plan and I moved to Render.

Contents

- Chapter 1: Data understanding and versioning (DVC)
- Chapter 2: Model development with TensorFlow/Keras
- Chapter 3: Experiment tracking with MLflow and DagsHub
- Chapter 4: Flask web application
- Chapter 5: Database trail
- Chapter 6: Containerization with Docker, Compose and CI
- Chapter 7: API deployment (Render)
- Chapter 8: Integrating Flask with the cloud API
- Chapter 9: Summary and limitations

Every screenshot is captioned and referenced in the text. Where the evidence for a step is a file, a setting or a result that lives in the repository rather than a screenshot, a short note in a blue box explains what was done and where to find it. The table below shows where each kind of evidence is kept.

Evidence	Where to find it
Data audit, EDA, splits, training, tuning, MLflow, registry and TFLite scripts	src/ in the GitHub repository
Dataset pointer and split lists	.dvc/, data/raw.dvc, data/splits/ (images in DagsHub DVC storage)
Experiments, comparison, model registry	DagsHub project: Experiments and Models tabs
Flask app (and database model), cloud API	app/, run.py; api/
Container files	Dockerfile, docker-compose.yml, .dockerignore, .env.example
Continuous integration, tests and overview	.github/workflows/ci.yml, tests/, Actions tab, README.md (CI badge)

Chapter 1: Data understanding and versioning

1.1 Environment

I created an isolated virtual environment (`.venv`) inside the project folder and installed TensorFlow and the supporting libraries into it. Isolating the environment means the project's packages cannot clash with anything else on my machine, and the exact versions can be recorded so someone else can recreate them. Figure 1.1 lists the installed versions: Python 3.13.5, TensorFlow 2.21.0, Keras 3.15.1, NumPy 2.5.4, pandas 3.0.6, Pillow 13.0.0, Flask 3.1.3, MLflow 3.17.1, DVC 3.67.1 and Keras Tuner 1.4.8. The full list is saved in `requirements-full.txt`.

TensorFlow could not use a GPU. From version 2.11 TensorFlow no longer supports GPUs on native Windows, and it says so itself when the model loads (the warning is visible in Figure 3.6b and Figure 4.1). All training therefore ran on the CPU, which is why one epoch of the baseline CNN took roughly 40 to 56 seconds (Figure 2.4a).

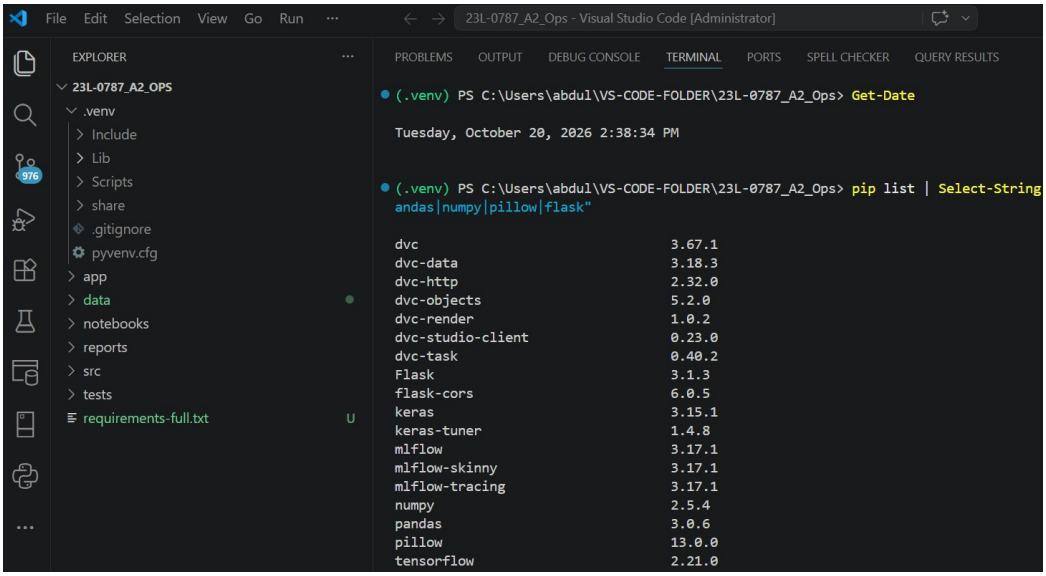


Figure 1.1: *Terminal in the project folder showing the date and the installed package versions.*

1.1 Environment check. The environment was created and checked as described above. Figure 1.1 and the file requirements-full.txt in the repository give the exact packages. TensorFlow reports in the logs of Figures 3.6b and 4.1 that it has no GPU support on native Windows, so every run used the CPU.

1.2 Dataset audit

I wrote my own script, `src/audit.py`, that walks the dataset folders and builds one table row per image: class (the folder name), file size, width, height, colour mode, format and a SHA-256 hash of the file bytes. An image counts as unreadable if Pillow's `verify()` fails. Two files are exact duplicates if their hashes are identical, because a hash is a fingerprint of the bytes. The table is saved as `data/audit.csv` and is the source for the train/validation/test lists later.

	AiArtData (AI-generated)	RealArt (real)
Images	539	436
Width min / median / max (px)	225 / 1024 / 6000	183 / 920 / 8495
Height min / median / max (px)	135 / 800 / 4001	158 / 720 / 7091
File size min / median / max (bytes)	11,738 / 199,760 / 5,626,183	11,311 / 144,840 / 10,195,352

Total: 975 images, **0 corrupt files** and **1 exact duplicate**. The colour modes are mixed: RGB 896, RGBA 49, palette (P) 23 and greyscale (L) 7. This matters for the input pipeline, because a model needs three channels, so every file must be converted to RGB when it is decoded. The classes are fairly balanced (about 55 percent AI). The single duplicate was removed before splitting, so the same image cannot sit in both the training and

the test set.

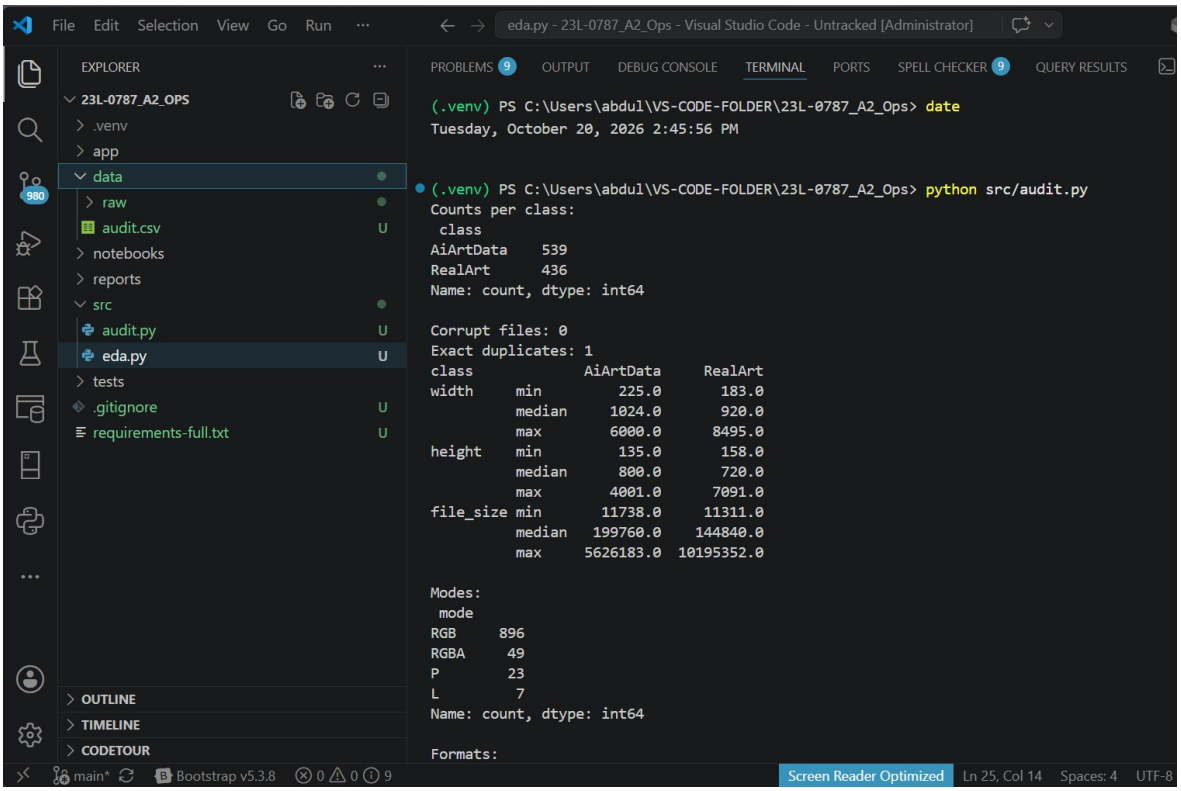


Figure 1.2: *Output of the audit script: counts per class, corrupt files, duplicates, and min/median/max of width, height and file size per class.*

1.3 Visual EDA

`src/eda.py` produces a sample grid and three histograms (Figures 1.3a to 1.3d).

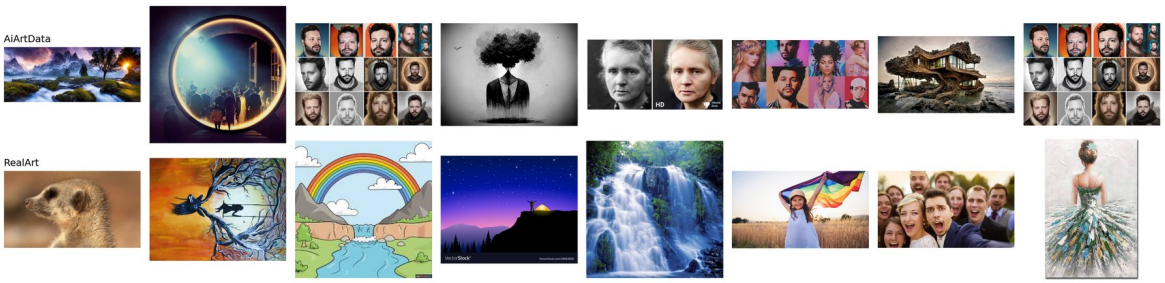


Figure 1.3a: *Sample images per class. The AI class (top) holds fantasy landscapes, surreal scenes and many portrait collages. The real class (bottom) holds photographs but also human-made paintings and cartoons.*

Interpretation. The real class is not only photographs: it also contains paintings, a vector illustration and a cartoon. So the task is to separate AI art from human-made images of all kinds, which is harder than photo versus AI. Several images are collages or carry watermarks, which the model could pick up as shortcuts.

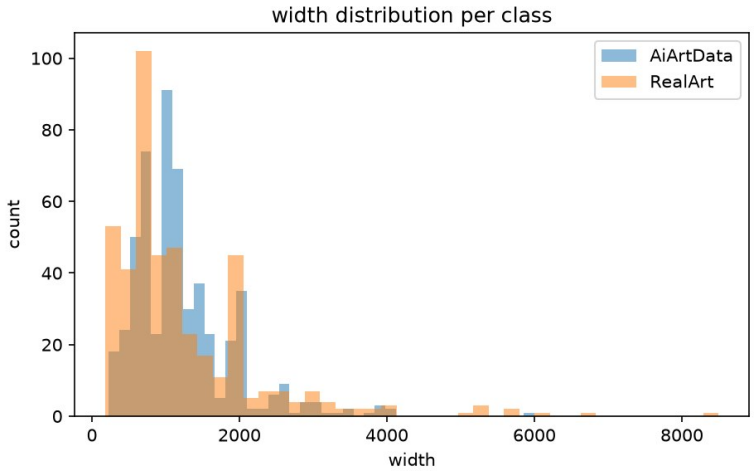


Figure 1.3b: *Histogram of image width per class.*

Interpretation. Both classes peak between about 700 and 1,200 px. AI images show sharp peaks at common generator output sizes (around 1,000 and 2,000 px), while real images are more spread out with a long tail up to 8,495 px. The overlap is large but not complete, so resolution could leak a little class information.

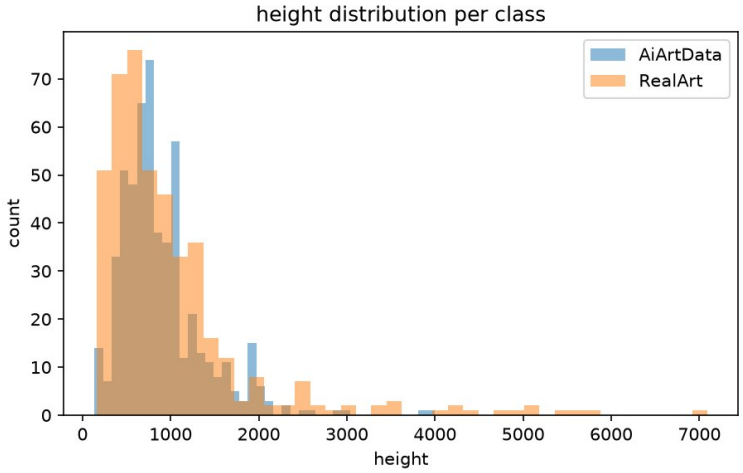


Figure 1.3c: *Histogram of image height per class.*

Interpretation. AI images concentrate around 800 to 1,000 px and stop near 4,001 px. Real images reach 7,091 px and have many more small and very tall files. The same caution applies as for width.

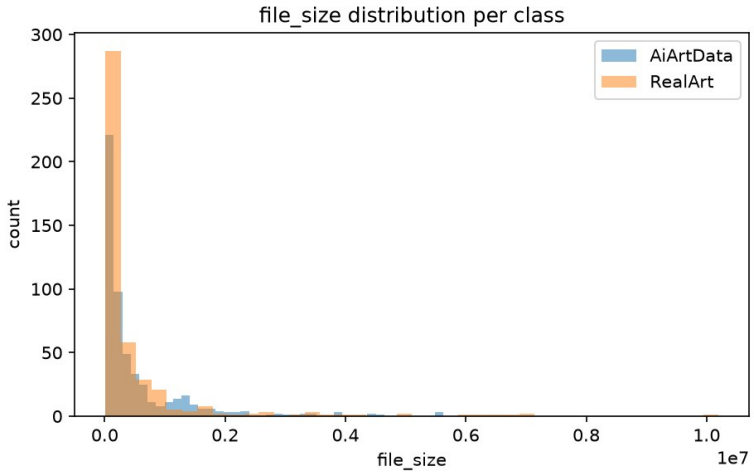


Figure 1.3d: *Histogram of file size per class (the axis is in units of 10 million bytes).*

Interpretation. Both distributions are strongly right-skewed, with most files below about 0.5 MB. Real images have more very small files and a longer tail up to about 10 MB, while the largest AI file is about 5.6 MB.

Compression and source differ between the classes, which is another possible shortcut. Resizing every image to the same square size (Chapter 2) removes the explicit size information, although the stretched aspect ratio can still hint at the original shape.

1.4 DVC (data version control)

Toy part. The exercise in a new empty folder (track a small CSV, change it, commit both versions, return to the older version, push to a remote, delete the data and pull it back) is documented below.

> **1.4 Toy DVC exercise.** In a new empty folder I ran git init and dvc init, created a small CSV and tracked it with dvc add, and committed the pointer file (version 1). I then appended rows, ran dvc add again and committed version 2, and went back to version 1 by checking out the older pointer file and running dvc checkout, which restored the original file. I set a local folder as the DVC remote, ran dvc push, deleted the CSV and the local cache, and ran dvc pull, which brought the data back. The pointer file (data.csv.dvc) holds only the MD5 hash, the size and the path of the CSV.

Applied to the dataset. In the project I ran `dvc init` and `dvc add data/raw`, which moved the data into the DVC cache and created the small pointer file `data/raw.dvc`. I added DagsHub's DVC storage as the remote (`.dvc/config` contains the remote URL; the username and token are stored with `--local` in `.dvc/config.local`, which Git ignores), committed the pointer to Git and ran `dvc push`. Figure 1.4a shows the folder being added and the DagsHub page listing the two class folders as DVC-tracked, and Figure 1.4b shows the push result.

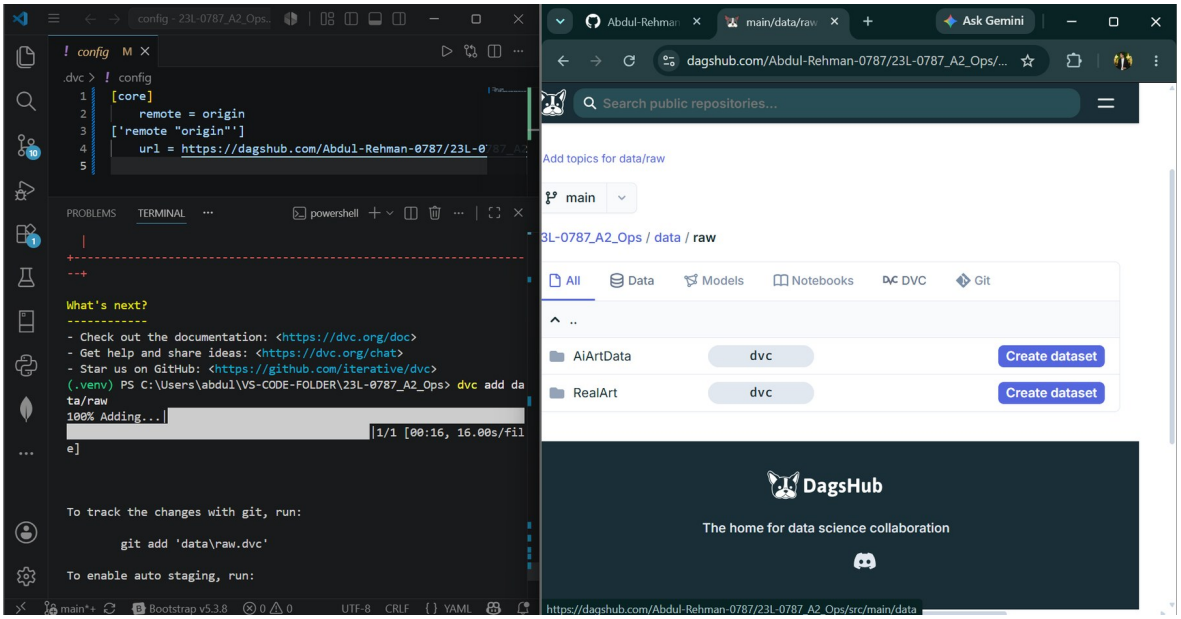


Figure 1.4a: *dvc add data/raw and the DagsHub data page showing AiArtData and RealArt tracked by DVC.*

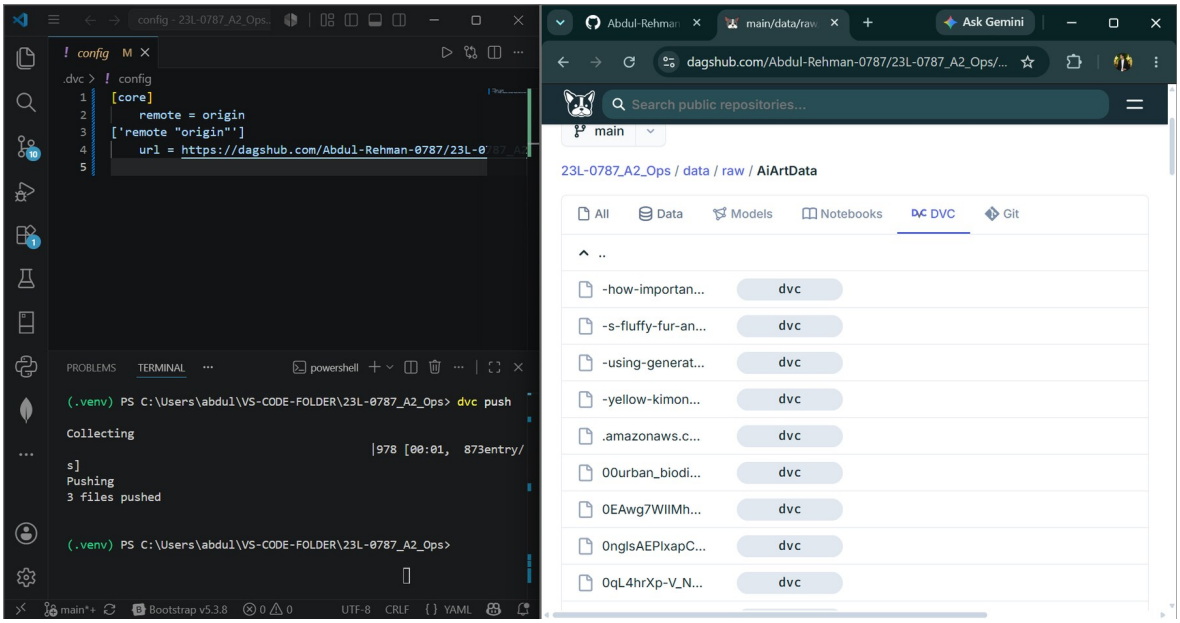


Figure 1.4b: *dvc push to the DagsHub remote, and the image files visible in DagsHub storage.*

What the pointer file holds. The pointer file `data/raw.dvc` is a few lines of text. It holds the MD5 hash of the dataset (a hash of a listing of every file and its own hash), the total size, the number of files and the path. Git stores only this tiny file. The real images live in the DVC cache and remote storage and are fetched with `dvc pull`. Large data does not belong in Git because Git keeps every version of every file in its history, so the repository would grow enormous and every clone would download it all. Git cannot diff images usefully, and hosting services limit file sizes. With DVC, checking out an older commit and running `dvc checkout` gives back the matching version of the data.

What went wrong. My first `dvc push` failed with "no remote specified" because I had added the remote but not made it the default; running it with `-r origin` (and later `dvc remote default origin`) fixed that. The next pushes failed with "Timeout on reading data from socket" and WinError 121 because many files were uploaded in parallel over an unstable connection. Re-running the push resumes and uploads only what is missing, and it finished with "2 files pushed". Finally, my first DagsHub repository was not linked to GitHub (its mirror field stayed empty), so I deleted it, recreated it with "Connect a repository" (Figure 3.5a shows the "Connected to" GitHub line) and pushed the data again.

Chapter 2: Model development with TensorFlow/Keras

2.0 Data preparation and splits

`src/make_splits.py` removes corrupt files and exact duplicates, assigns label 1 to AiArtData (the positive class) and label 0 to RealArt, and makes a stratified 70/15/15 split with a fixed random seed (42). Stratifying keeps the class ratio equal in every split, and the fixed seed makes the split reproducible. The three CSV lists are committed to Git (`data/splits/`).

Split	Images	Share that is AI
Train	681	0.552
Validation	146	0.555
Test	147	0.551

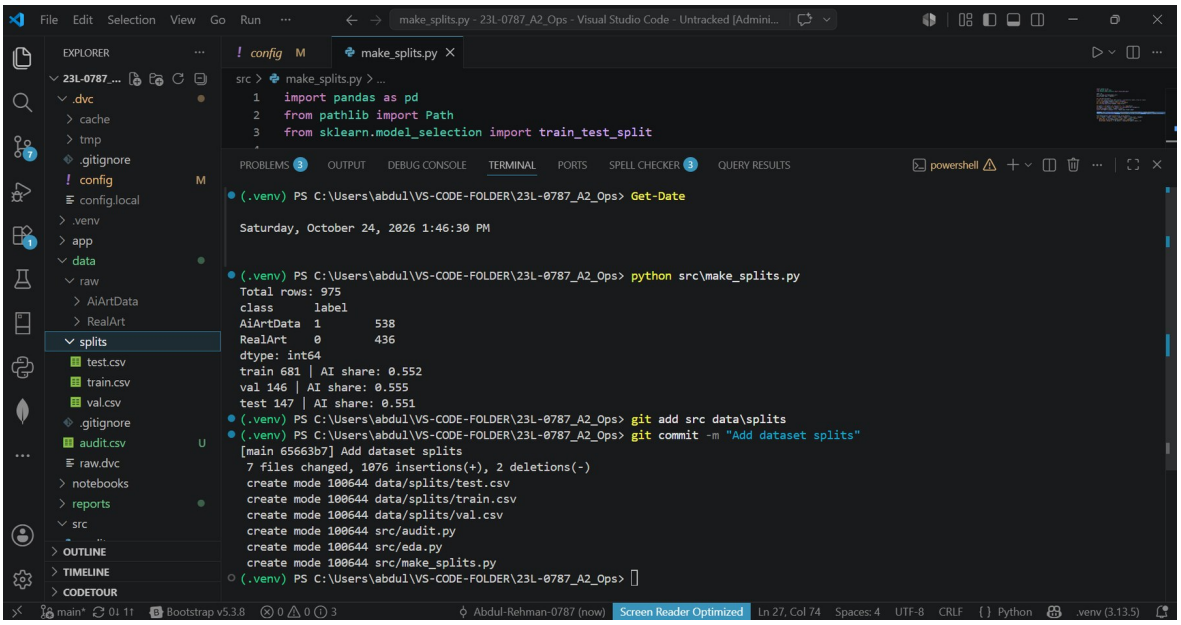


Figure 2.0: *Output of make_splits.py: 975 rows before cleaning (538 AI and 436 real after removing the duplicate) and the three splits.*

2.1 Input pipeline

The `tf.data` pipeline in `src/common.py` works in these stages. (1) It reads the split list (file path and label). (2) For the training split only, it shuffles with a fixed seed. (3) It reads each file and decodes it with `decode_image(channels=3)`, which turns grayscale, palette and RGBA files into RGB. (4) It resizes to 224 x 224. (5) It batches 32 images. (6) For the training split only, it applies random augmentation: horizontal flip, rotation up to 8 percent of a full turn, zoom up to 10 percent and contrast change up to 10 percent. (7) It prefetches, so the next batch is prepared while the model trains.

I chose 224 x 224 because it is the size MobileNetV2 was pretrained on and it keeps CPU training affordable. **Aspect ratio:** `tf.image.resize` stretches the image to a square without preserving its aspect ratio, so a wide image is squeezed horizontally. Padding would add black bars and cropping would lose content, so I accepted the distortion for simplicity, but it is a known limitation for extremely wide or tall images. Pixels stay in the range 0 to 255 in the pipeline, because the scaling is done inside the model (this keeps serving simple). Augmentation is applied to training data only, because validation and test scores must measure the model on unmodified images.

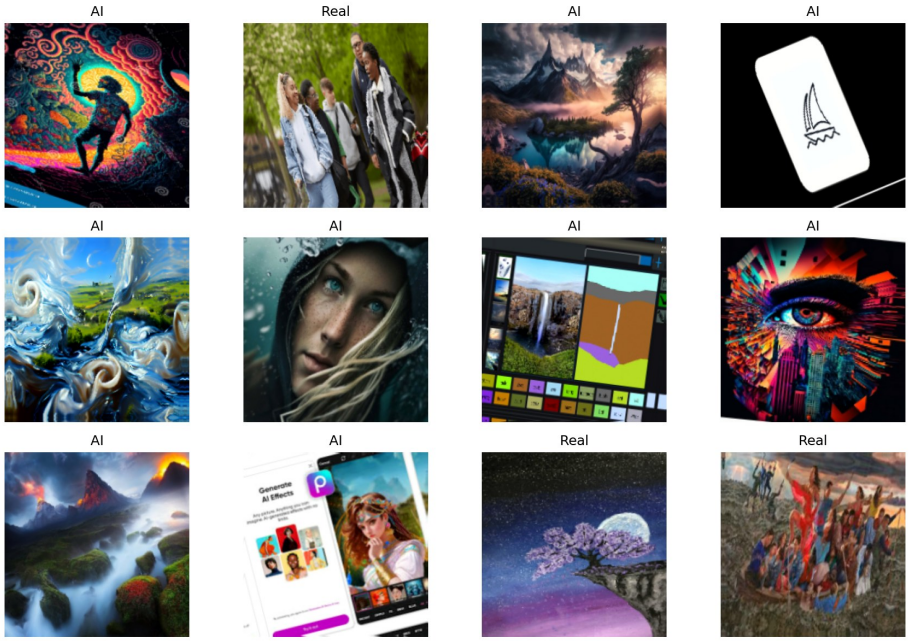


Figure 2.1: *One augmented training batch (12 images). Some images are rotated, zoomed or changed in contrast; the label (AI or Real) is shown above each image.*

2.2 Baseline CNN

The baseline is a small CNN written with Keras layers (`src/models.py`). Each block has a convolution, batch normalization, ReLU and max pooling, repeated with 32, 64 and 128 filters, followed by global average pooling, dropout and one sigmoid output. The table lists each layer.

Layer	Output shape	Parameters
Input	(224, 224, 3)	0
Rescaling (divide by 255)	(224, 224, 3)	0
Conv2D 32 filters, 3x3	(224, 224, 32)	864
BatchNormalization	(224, 224, 32)	128
ReLU, then MaxPooling2D	(112, 112, 32)	0
Conv2D 64 filters, 3x3	(112, 112, 64)	18,432
BatchNormalization	(112, 112, 64)	256
ReLU, then MaxPooling2D	(56, 56, 64)	0
Conv2D 128 filters, 3x3	(56, 56, 128)	73,728
BatchNormalization	(56, 56, 128)	512
ReLU, then MaxPooling2D	(28, 28, 128)	0
GlobalAveragePooling2D	(128)	0
Dropout 0.4	(128)	0
Dense 1, sigmoid	(1)	129
Total		**94,049** (448 are batch-norm moving statistics, not trained)

The first rows of the table are visible in Figure 2.2; the rows after the third ReLU come from the model definition and the standard parameter formulas, because the screenshot is cut off.

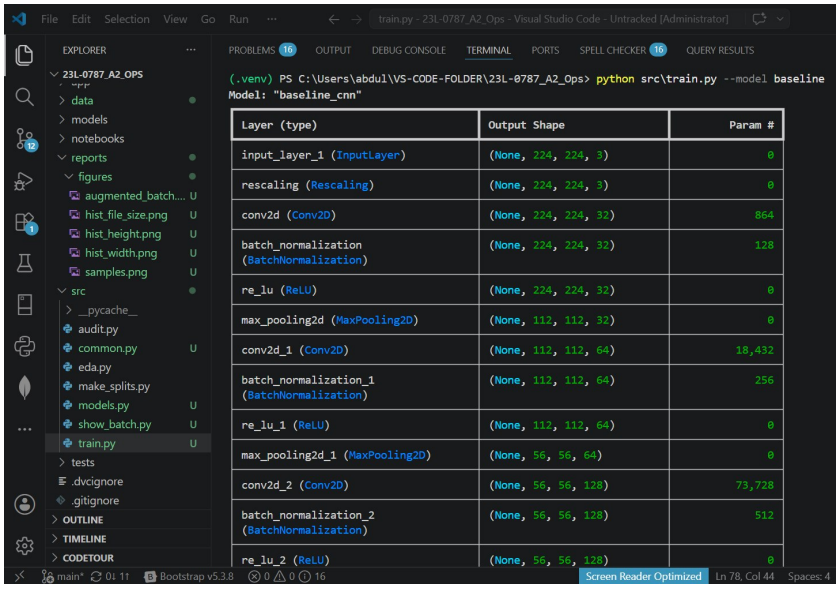


Figure 2.2: *Keras model summary of the baseline CNN (first layers).*

Why each layer. Rescaling maps pixels to the 0 to 1 range so training is stable. Convolutions learn local patterns (edges, textures). Batch normalization stabilises and speeds up training (and the convolutions need no bias). ReLU adds non-linearity. Max pooling halves the spatial size, keeping computation low and giving some tolerance to small shifts. Global average pooling replaces a huge flatten layer, which means far fewer parameters and less overfitting on a small dataset. Dropout randomly switches units off so the model cannot memorise the training set. The single sigmoid unit outputs the probability that the image is AI-generated.

Loss, optimizer, metrics. Binary cross-entropy is the standard loss for a sigmoid output with two classes. Adam (learning rate 1e-3) adapts the step size for each weight, which works well without much tuning. I track accuracy and AUC; AUC does not depend on the decision threshold, so it shows how well the classes are ranked.

2.3 Transfer learning

I loaded MobileNetV2 from `keras.applications` with ImageNet weights and without its classifier, then added my own head: global average pooling, dropout 0.3 and a one-unit sigmoid layer. **Preprocessing:** MobileNetV2 was pretrained on pixels scaled to the range -1 to 1, so my model starts with a Rescaling layer that computes x / 127.5 - 1. With the wrong range the pretrained features would be misused and much of the benefit of transfer learning lost. The backbone is called with `training=False` so its batch-normalization layers keep their pretrained statistics.

Stage 1 freezes the whole backbone and trains only the head with learning rate 1e-3. Figure 2.3a shows that only the layer `dense` is trainable: 1,281 trainable parameters out of 2,259,265 (the other 2,257,984 are frozen). **Stage 2** unfreezes only the last block of the backbone (the layers whose names start with `block_16` and `Conv_1`, keeping batch-normalization layers frozen) and continues with a much smaller learning rate of 1e-5, so the pretrained weights are adjusted gently instead of being destroyed. The training curves for both stages are in Figures 2.4d and 2.4e. Stage 1 reached its best validation loss at epoch 28 and stage 2 at epoch 7 (validation loss 0.4778). I kept the stage 2 model as the final transfer model because it had the lower validation loss.

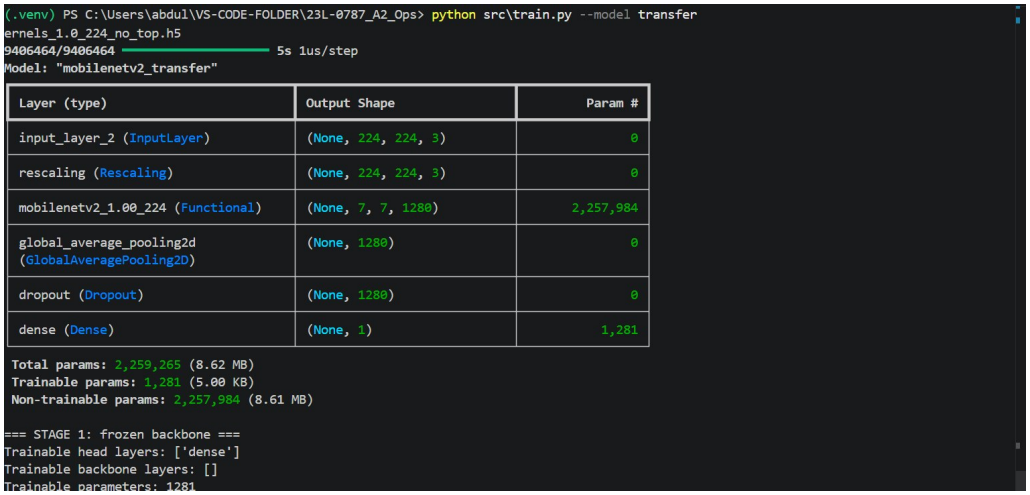


Figure 2.3a: *MobileNetV2 transfer model summary and the stage 1 trainable layers (only dense).*

2.3 Stage 2 trainable layers. In stage 2 the function unfreeze_last_block (src/models.py in the repository) makes the layers of the last MobileNetV2 block trainable (the block_16 layers and Conv_1) and keeps every batch-normalization layer frozen. Everything before that block stays frozen.

2.4 Training discipline

I fixed all random seeds with `keras.utils.set_random_seed(42)`, which seeds Python, NumPy and TensorFlow. Three callbacks are used: EarlyStopping on validation loss (patience 6, restoring the best weights), ModelCheckpoint saving the best model by validation loss, and ReduceLROnPlateau halving the learning rate when validation loss stalls (patience 3). The best epochs were: baseline 29 (validation loss 0.5901), transfer stage 1 epoch 28, and transfer stage 2 epoch 7 (validation loss 0.4778).

- **Baseline:** it never triggered early stopping and ran all 30 epochs. The learning rate was halved to 2.5e-4 (Figure 2.4a) and the validation loss was still improving, so it is under-trained rather than overfitted.
- **Stage 1:** validation loss flattens around 0.50 while training loss keeps falling, which shows that a frozen backbone limits what the head can learn.
- **Stage 2:** validation loss stays near 0.48 while training loss falls to about 0.27 and training accuracy reaches about 0.90 against about 0.79 on validation. That gap is the start of overfitting, and early stopping ended the run at epoch 13 (best epoch 7 plus patience 6).

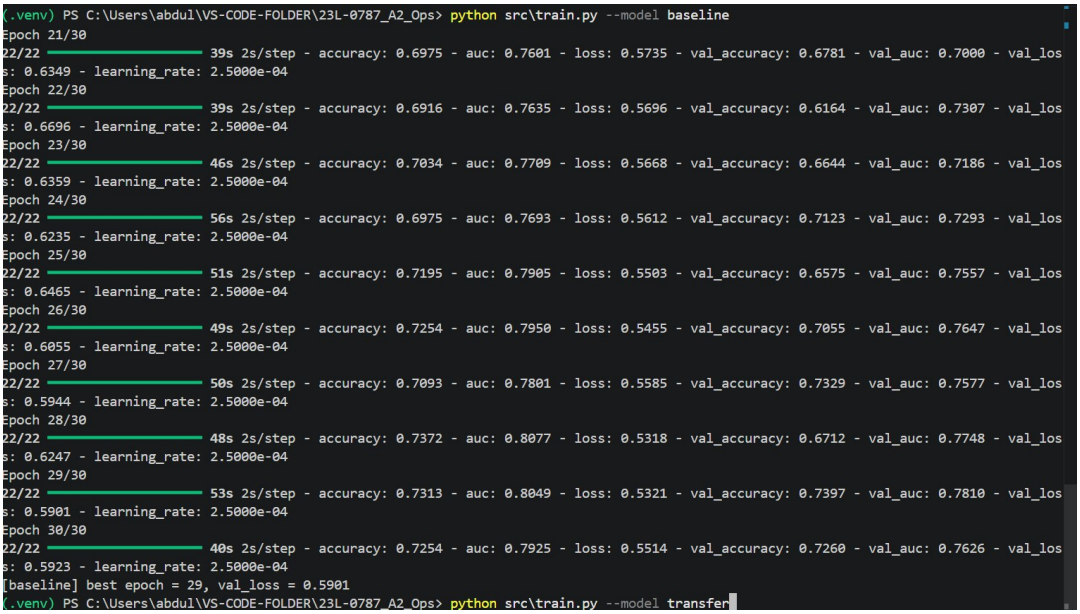


Figure 2.4a: *End of baseline training: epochs 21 to 30, the reduced learning rate, and the line "best epoch = 29, val_loss = 0.5901".*

```
(.venv) PS C:\Users\abdul\VS-CODE-FOLDER\23L-0787_A2_Ops> python src\train.py --model transfer
22/22 ━━━━━━━━━━━━━━━━━━━━ 17s 716ms/step - accuracy: 0.8399 - auc: 0.9331 - loss: 0.3324 - val_accuracy: 0.8014 - val_auc: 0.8550 - val_
loss: 0.4914 - learning_rate: 1.0000e-05
Epoch 5/30
22/22 ━━━━━━━━━━━━━━━━━━━━ 17s 693ms/step - accuracy: 0.8590 - auc: 0.9395 - loss: 0.3223 - val_accuracy: 0.7877 - val_auc: 0.8531 - val_
loss: 0.4900 - learning_rate: 1.0000e-05
Epoch 6/30
22/22 ━━━━━━━━━━━━━━━━━━━━ 16s 679ms/step - accuracy: 0.8576 - auc: 0.9388 - loss: 0.3237 - val_accuracy: 0.7877 - val_auc: 0.8590 - val_
loss: 0.4959 - learning_rate: 1.0000e-05
Epoch 7/30
22/22 ━━━━━━━━━━━━━━━━━━━━ 16s 665ms/step - accuracy: 0.8796 - auc: 0.9502 - loss: 0.2941 - val_accuracy: 0.8014 - val_auc: 0.8582 - val_
loss: 0.4778 - learning_rate: 5.0000e-06
Epoch 8/30
22/22 ━━━━━━━━━━━━━━━━━━━━ 16s 640ms/step - accuracy: 0.8737 - auc: 0.9487 - loss: 0.3031 - val_accuracy: 0.7945 - val_auc: 0.8582 - val_
loss: 0.4790 - learning_rate: 5.0000e-06
Epoch 9/30
22/22 ━━━━━━━━━━━━━━━━━━━━ 16s 676ms/step - accuracy: 0.8840 - auc: 0.9528 - loss: 0.2891 - val_accuracy: 0.7945 - val_auc: 0.8588 - val_
loss: 0.4801 - learning_rate: 5.0000e-06
Epoch 10/30
22/22 ━━━━━━━━━━━━━━━━━━━━ 16s 694ms/step - accuracy: 0.8767 - auc: 0.9473 - loss: 0.2992 - val_accuracy: 0.7877 - val_auc: 0.8576 - val_
loss: 0.4801 - learning_rate: 5.0000e-06
Epoch 11/30
22/22 ━━━━━━━━━━━━━━━━━━━━ 16s 677ms/step - accuracy: 0.8913 - auc: 0.9515 - loss: 0.2906 - val_accuracy: 0.7945 - val_auc: 0.8572 - val_
loss: 0.4798 - learning_rate: 2.5000e-06
Epoch 12/30
22/22 ━━━━━━━━━━━━━━━━━━━━ 16s 642ms/step - accuracy: 0.8943 - auc: 0.9600 - loss: 0.2748 - val_accuracy: 0.7877 - val_auc: 0.8586 - val_
loss: 0.4801 - learning_rate: 2.5000e-06
Epoch 13/30
22/22 ━━━━━━━━━━━━━━━━━━━━ 16s 687ms/step - accuracy: 0.8957 - auc: 0.9592 - loss: 0.2735 - val_accuracy: 0.7877 - val_auc: 0.8603 - val_
loss: 0.4803 - learning_rate: 2.5000e-06
[transfer_stage2] best epoch = 7, val_loss = 0.4778
Final transfer model = transfer_stage2
(.venv) PS C:\Users\abdul\VS-CODE-FOLDER\23L-0787_A2_Ops>
```

Figure 2.4b: *End of transfer stage 2: best epoch 7, validation loss 0.4778, and the final transfer model chosen.*

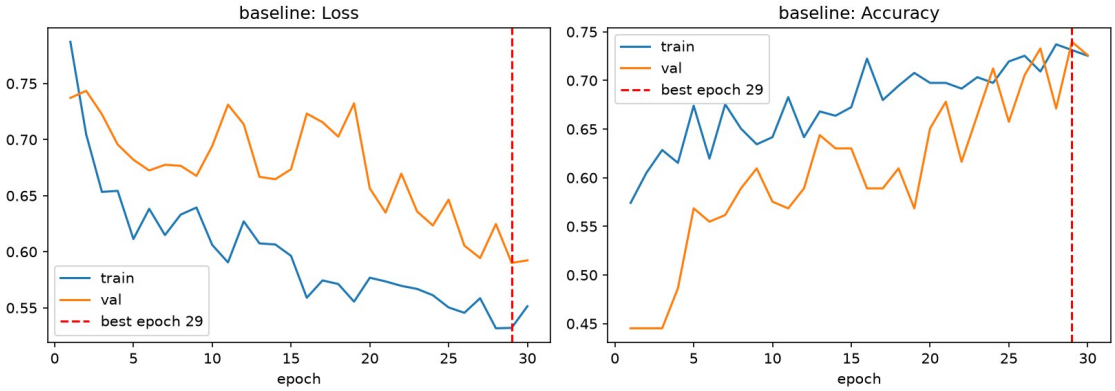


Figure 2.4c: *Baseline loss and accuracy curves with the best epoch (29) marked.*

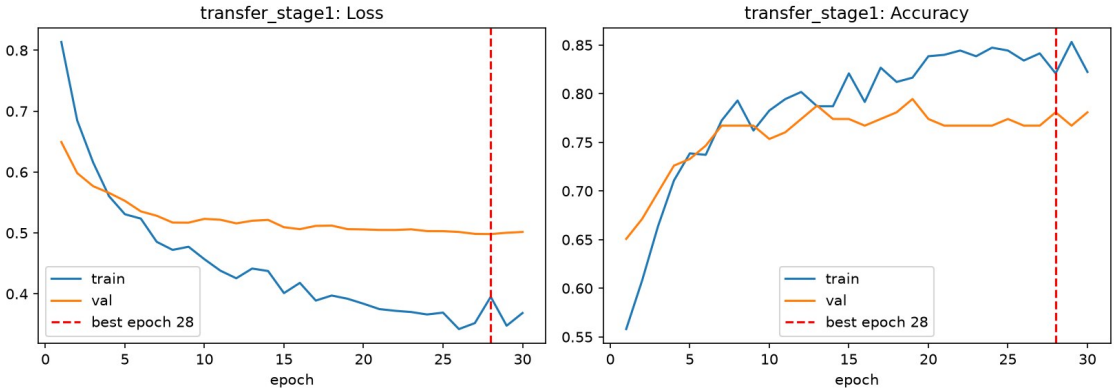


Figure 2.4d: *Transfer stage 1 (frozen backbone) curves, best epoch 28.*

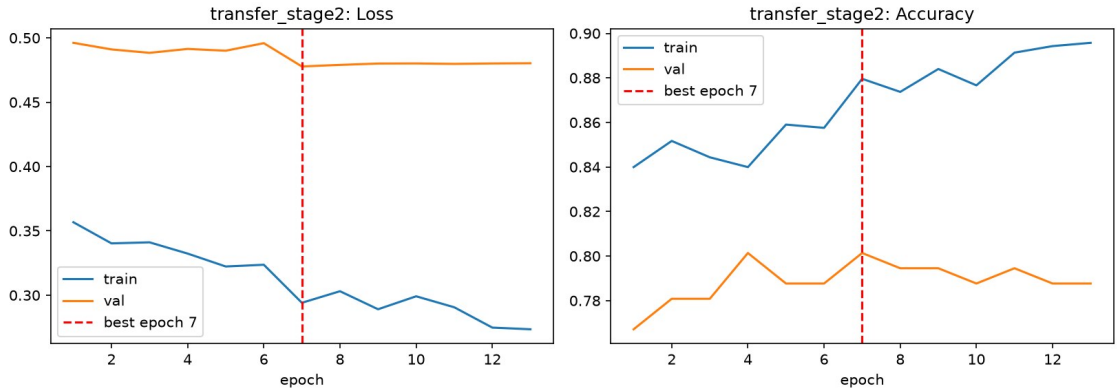


Figure 2.4e: *Transfer stage 2 (last block unfrozen) curves, best epoch 7.*

2.5 Evaluation

`src/evaluate.py` first chooses the decision threshold on the **validation** set by scanning thresholds from 0.05 to 0.95 and maximising an F-beta score, and then evaluates the test set once with that threshold. Choosing the threshold on the test set would make the test score optimistic, because the test set would have been used for tuning. The chosen thresholds were 0.47 for the baseline and 0.33 for the transfer model (Figures 2.5a and 2.5c). The metrics below come from the confusion matrices (AI is the positive class; 147 test images: 66 real, 81 AI) and the ROC legends.

Metric (test set)	Baseline CNN	MobileNetV2 transfer
Decision threshold (set on validation)	0.47	0.33
Accuracy	0.694	0.762
Precision	0.673	0.735
Recall	0.864	0.889
F1	0.757	0.804
ROC-AUC	0.715	0.865
Real images kept as real (specificity)	32 of 66 (0.485)	40 of 66 (0.606)
Confusion matrix: TN / FP / FN / TP	32 / 34 / 11 / 70	40 / 26 / 9 / 72

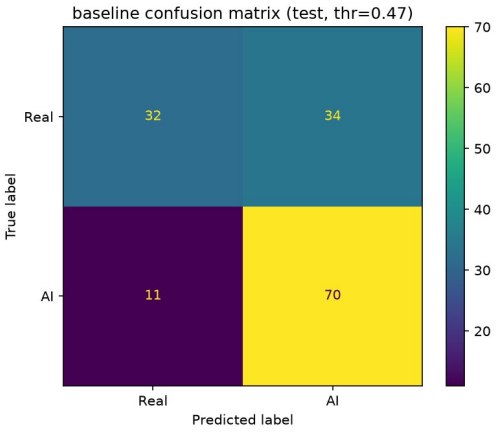


Figure 2.5a: *Baseline confusion matrix on the test set (threshold 0.47).*

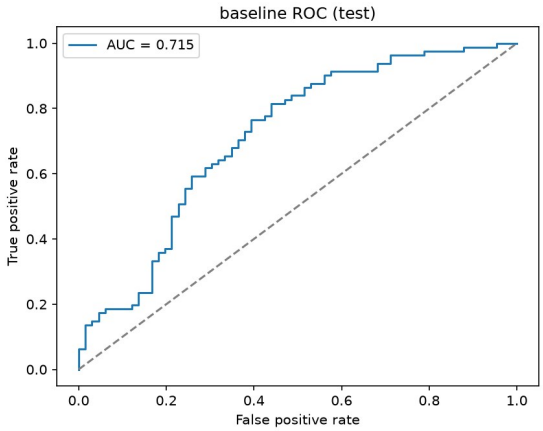


Figure 2.5b: *Baseline ROC curve on the test set (AUC 0.715).*

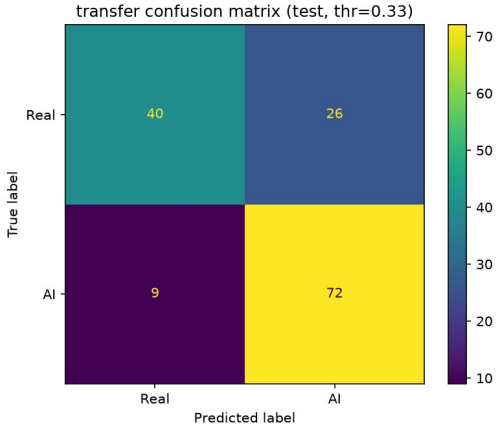


Figure 2.5c: *Transfer model confusion matrix on the test set (threshold 0.33).*

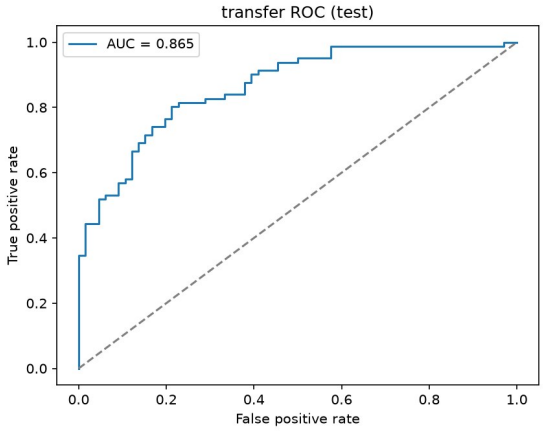


Figure 2.5d: *Transfer model ROC curve on the test set (AUC 0.865).*

Which error is worse. Calling a real image fake wrongly accuses a creator, and calling a fake image real lets synthetic content pass as genuine. For a tool whose purpose is to catch AI images, I treat the missed fake as the more costly mistake, so I prefer a threshold that favours recall. Both thresholds are below 0.5, and the transfer model reaches recall 0.889 (9 of 81 AI images missed). The price is 26 real images flagged as AI out of 66, so the product must present its answer as an estimate that invites a second look, not as a final accusation. The transfer model is better than the baseline on every measure: ROC-AUC rises from 0.715 to 0.865 and both kinds of error fall. With only 147 test images the accuracy has an uncertainty of about 7 percentage points (95 percent), so small differences should not be over-interpreted.

2.6 Error analysis

Figure 2.6a shows the ten most confident mistakes of the transfer model on the test set (the full list is in Figure 2.6b). Overall there were 35 errors (26 false alarms and 9 misses). The hypotheses below are my reading of the images; I did not test them.

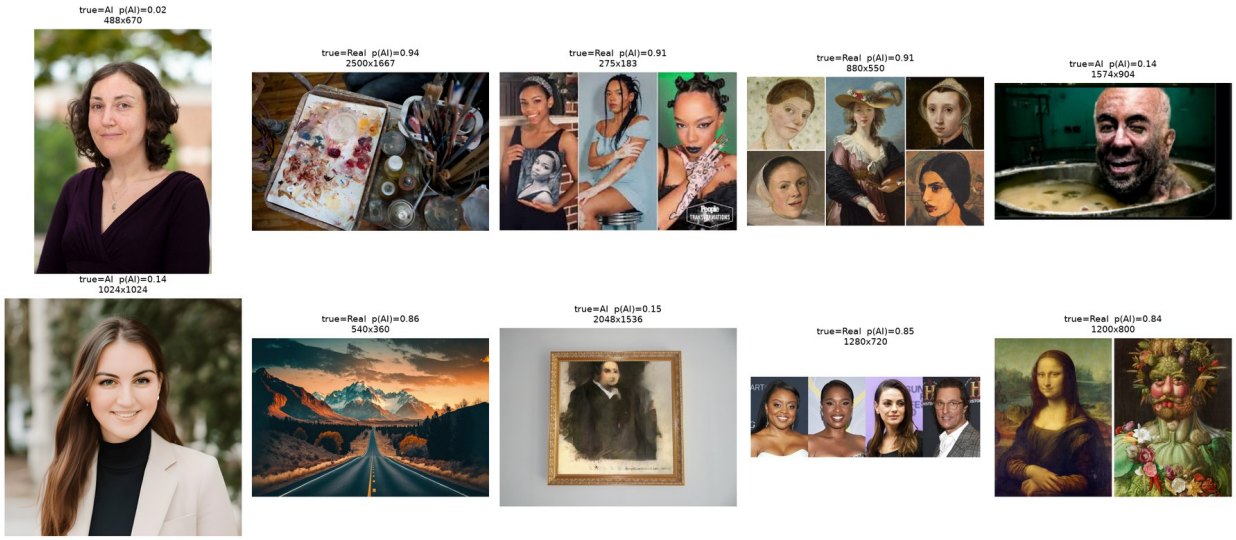


Figure 2.6a: *Ten most confident misclassified test images, in the order of the table below (left to right, top to bottom).*

```
(.venv) PS C:\Users\abdul\VS-CODE-FOLDER\23L-0787_A2_Ops> python src\errors.py transfer
                                                path  label      prob  width  height  file_size
26                 data/raw/AiArtData/NeseDevenot_02.jpg      1  0.017084    488     670      36683
31                         data/raw/RealArt/_MG_5812.jpg      0  0.938157   2500    1667     890920
6                          data/raw/RealArt/images45.jpg      0  0.913462    275     183      11769
5                data/raw/RealArt/3-x-3-template-newi.jpg      0  0.908446    880     550     126299
7                     data/raw/AiArtData/Midjourney-2.png      1  0.135921   1574     904    1008197
0                           data/raw/AiArtData/alice.png      1  0.139985   1024    1024    1660533
3   data/raw/RealArt/360_F_568525285_CrGJ8Yh1noTyq...      0  0.856242    540     360      63058
16  data/raw/AiArtData/merlin_145496349_2ef618d1-e...      1  0.147064   2048    1536     457909
30                          data/raw/RealArt/people.jpg      0  0.850990   1280     720     132081
10  data/raw/RealArt/famous-portrait-paintings_wid...      0  0.836365   1200     800    1750322
(.venv) PS C:\Users\abdul\VS-CODE-FOLDER\23L-0787_A2_Ops>
```

Figure 2.6b: *Printed table of the same ten errors (path, label, probability, size).*

#	True / p(AI)	Size	Hypothesis
1	AI / 0.02	488x670	Photorealistic headshot with natural light and blurred background. It looks like a stock portrait, not like the colourful fantasy art the model associates with AI.
2	Real / 0.94	2500x1667	Overhead photo of a paint palette: dense, saturated colour clutter and fine texture resemble AI-art detail.
3	Real / 0.91	275x183	Tiny three-photo collage with a watermark. Upscaling to 224 px blurs it and the collage layout is atypical.
4	Real / 0.91	880x550	Collage of old painted portraits. Painterly faces and grid layouts look like the AI portrait grids in the other class.
5	AI / 0.14	1574x904	Dark, grainy, low-saturation surreal face. It lacks the bright, saturated look typical of AI images in this data.
6	AI / 0.14	1024x1024	Another photorealistic portrait (smiling woman, blazer), like error 1.
7	Real / 0.86	540x360	Strongly colour-graded mountain road that looks like AI art. Either heavy editing or a possible label error.
8	AI / 0.15	2048x1536	Photograph of a framed painting on a wall. The photographic frame and wall dominate the image.
9	Real / 0.85	1280x720	Collage of four celebrity face photos. Face grids are common in the AI class.
10	Real / 0.84	1200x800	Classical paintings (a portrait and a surreal fruit-and-vegetable face). Painterly style is mistaken for AI style.

Pattern. The six false alarms are painted, collaged or heavily stylised real images, and the four misses are photorealistic or dark, low-saturation AI images. The model seems to rely on style (saturation, painterly texture,

collage layouts) rather than on real evidence of synthesis, which is consistent with the Grad-CAM maps below.

2.7 Tool lab: Keras Tuner

Toy part. I tuned a tiny dense network on Fashion-MNIST with random search, six trials of three epochs, searching the number of units (32 to 256) and the learning rate (1e-4 to 1e-2, log scale). The best trial used 128 units and learning rate 0.000307, with validation accuracy 0.8658.

Trial	Units	Learning rate	Validation accuracy
4	128	0.000307	0.865833
2	256	0.000505	0.865667
5	256	0.001715	0.862750
1	128	0.000350	0.862167
0	192	0.000119	0.858083
3	160	0.000136	0.856083

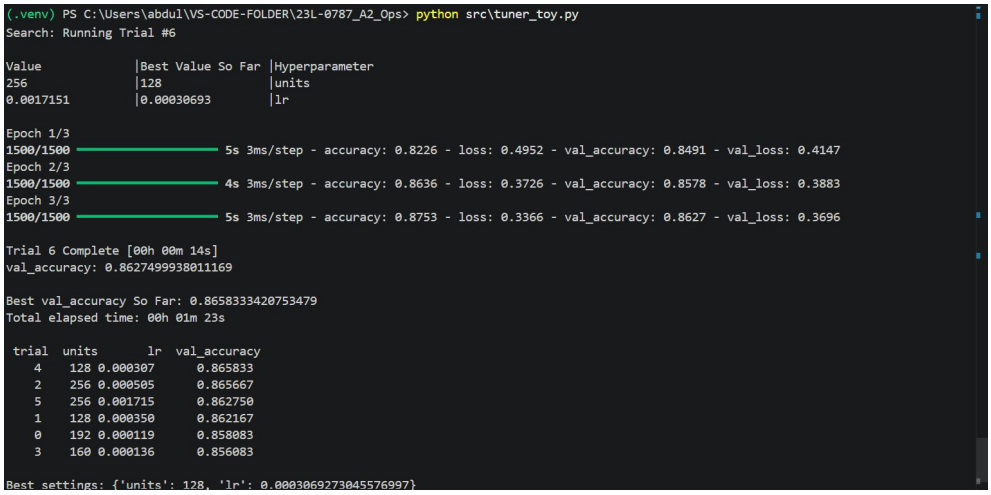


Figure 2.7a: *Toy tuner run: six trials, the results table and the best settings.*

VisionGuard. I tuned the learning rate (1e-5 to 1e-3, log scale), the dropout (0.2, 0.3, 0.5) and the number of unfrozen backbone layers (0, 10, 30, 60) of the MobileNetV2 model with random search: 10 trials of up to 6 epochs each, objective validation loss, total time 16 minutes 33 seconds. The best trial used 30 unfrozen layers, dropout 0.2 and learning rate 6.6e-5, with validation loss 0.425.

Trial	Unfrozen layers	Dropout	Learning rate	Validation loss
00	30	0.2	0.000066	0.425005
02	30	0.2	0.000099	0.426252
01	10	0.5	0.000051	0.468030
06	10	0.3	0.000718	0.471424
09	30	0.5	0.000044	0.473036
07	10	0.3	0.000068	0.522895
08	0	0.2	0.000304	0.601821
03	0	0.5	0.000172	0.657257
04	60	0.2	0.000855	0.687131
05	0	0.3	0.000028	0.755873

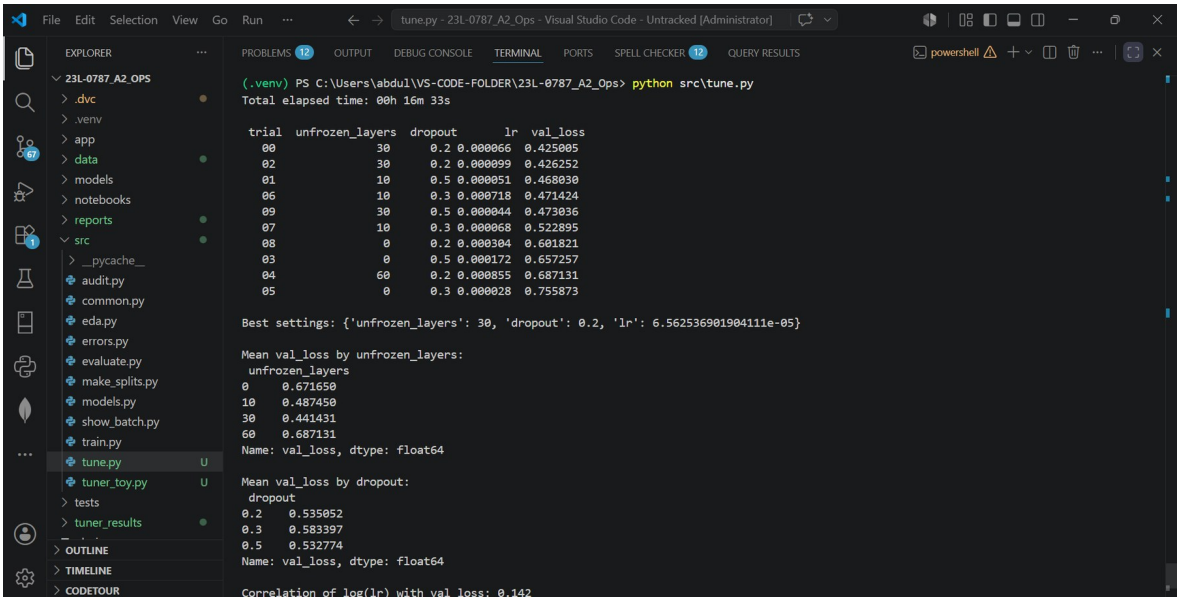


Figure 2.7b: *VisionGuard tuner: ranked trials, best settings, mean validation loss by unfrozen layers and by dropout, and the correlation of log learning rate with validation loss.*

Which setting mattered most. The number of unfrozen layers: mean validation loss was 0.67 with 0 unfrozen layers, 0.49 with 10, 0.44 with 30 and 0.69 with 60. Dropout had a narrow range of means (0.53 to 0.58) and the correlation of log learning rate with validation loss was only 0.14. Unfreezing lets the backbone adapt to this data (art styles differ from ImageNet photographs), but unfreezing too much on 681 training images is risky. These conclusions are tentative: only 10 random trials were run for three settings, and the effects are mixed. Trial 04 had 60 unfrozen layers *and* the largest learning rate (8.6e-4), so I cannot say which caused its poor result. The 0-layer trials trained only a head for six epochs, some at tiny learning rates (trial 05 used 2.8e-5), so they probably had not converged.

Why the best validation score can overfit the validation set. By trying 10 settings and keeping the best one on the same 146 validation images, I partly select for luck: some of the winning score is noise that happens to match those particular images, so 0.425 is an optimistic estimate. In both tables the top trials differ by only 0.001 to 0.003, which is smaller than the noise on such a small set. An untouched test set is needed for an honest final estimate, which is why the test set was used only once at the end.

Bonus: Grad-CAM

Grad-CAM shows which regions of an image most increased the model's score. Figure 2.8 shows four correct and four wrong test predictions of the transfer model (top row: image with its label and probability, bottom row: heat map; red means more influence).

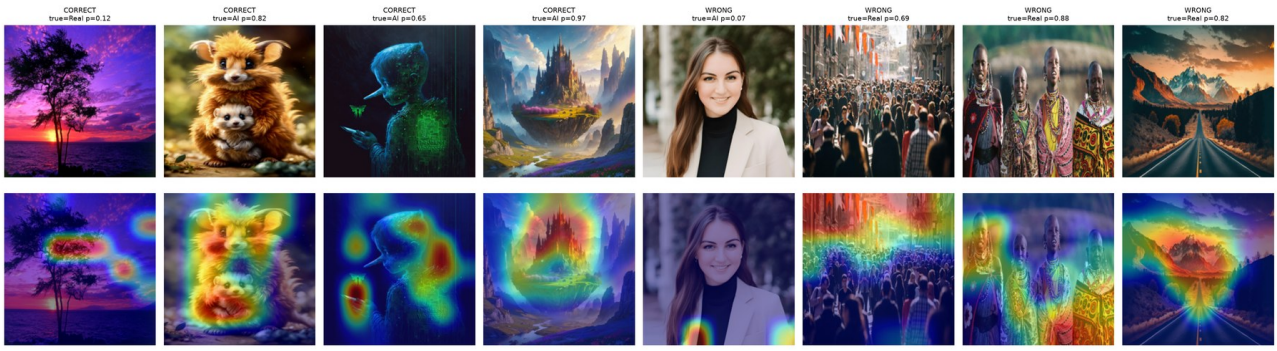


Figure 2.8: *Grad-CAM for 4 correct (left) and 4 wrong (right) predictions.*

- **Correct predictions:** the heat sits on the main subject (the creature, the castle, the character's head), which is a sensible place to look.
- **Wrong predictions:** for the AI portrait labelled real (p = 0.07) the heat is in the bottom corners and not on the face, so the model does not seem to use facial cues. For the real crowd, the real group portrait and the

real mountain road, the heat falls on detailed or dramatic regions (flags and buildings, patterned clothing, the peak), which look like what the model has learned to call AI.

- These maps are a visual aid, not proof of what the model "understands", but they support the idea that the model relies on style rather than on evidence of synthesis.

Chapter 3: Experiment tracking with MLflow and DagsHub

3.1 Tool lab: MLflow locally

I trained a tiny Keras network on MNIST (the first 20,000 images, one hidden layer of 64 units) twice, with learning rates 0.001 and 0.01. Each run logs the parameters (learning rate, epochs, units), the per-epoch metrics through a callback, one plot (the loss curve) and the model, in a local MLflow store (`mlflow_toy.db`). I opened the local MLflow interface at 127.0.0.1:5000 and compared the runs (Figure 3.1). The final accuracies were 0.96694 and 0.96838, so the learning rate made almost no difference after five epochs.

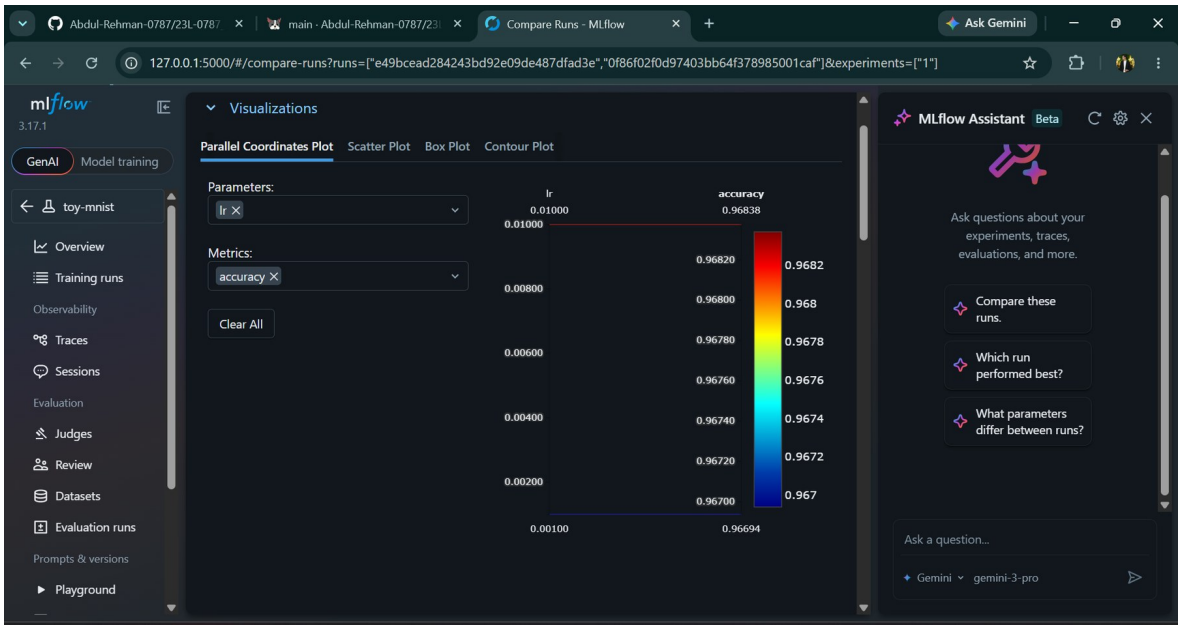


Figure 3.1: *MLflow 3.17.1 comparison view of the two toy runs (experiment toy-mnist), parallel coordinates of learning rate against accuracy.*

3.1 Run pages. Both toy runs (learning rates 0.001 and 0.01) were logged with their parameters, per-epoch metrics, the loss-curve plot and the model, and they appear side by side in the comparison view of Figure 3.1.

Definitions in my own words.

- **Experiment:** a named group of related runs, such as all my VisionGuard training attempts.
- **Run:** one execution of the training code, with its own parameters, metrics and files.
- **Parameter:** a setting I choose before the run and that stays fixed during it, such as the learning rate.
- **Metric:** a number the run measures, either per epoch (validation loss) or at the end (test F1).
- **Artifact:** a file produced by a run and stored with it, such as a plot, a split list or a model.
- **Tag:** a free label attached to a run to help searching and give context, such as the git commit or the seed.

3.2 DagsHub setup

The DagsHub project is connected to my GitHub repository (the line "Connected to https://github.com/Abdul-Rehman-0787/23L-0787_A2_Ops.git" at the top of Figure 3.5a). Its MLflow tracking address is `https://dagshub.com/Abdul-Rehman-0787/23L-0787_A2_Ops.mlflow`. I created an access token in the DagsHub settings and gave both to my code through the environment variables `MLFLOW_TRACKING_URI`, `MLFLOW_TRACKING_USERNAME` and `MLFLOW_TRACKING_PASSWORD`, which are stored in a local `.env` file and read by python-dotenv. MLflow picks them up on its own, so no secret appears in the code. The file `.env` is listed in `.gitignore`, and a `.env.example` with fake values is committed. I repeated the toy run against DagsHub; Figure 3.2 shows the five training epochs in the terminal and the run `lr_0.001` in the DagsHub Experiments tab.

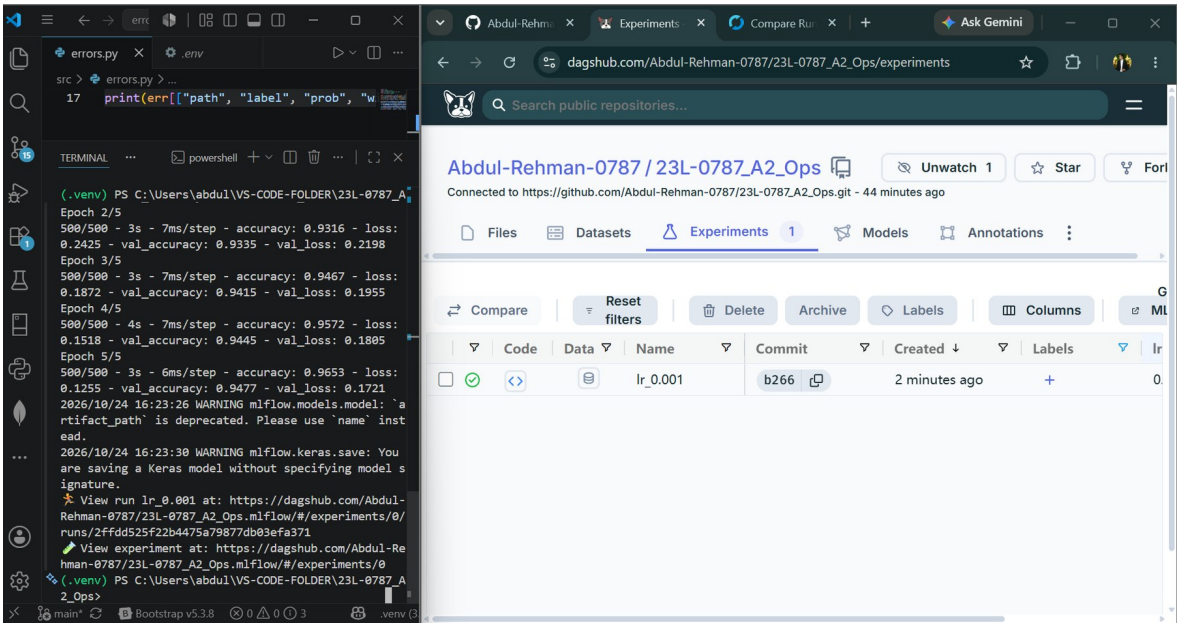


Figure 3.2: *Toy run logged to DagsHub: terminal output with the run URL (left) and the run in the DagsHub Experiments tab (right).*

3.2 Keeping secrets out of Git. The file .env is listed in .gitignore and does not appear in the GitHub repository, while .env.example (with placeholder values) does. The tracking address, username and token therefore stay on my machine and are read from environment variables.

MLflow 3 printed two harmless warnings (visible in Figure 3.2): the argument `artifact_path` is deprecated in favour of `name`, and the model was saved without a signature.

3.3 Instrumenting VisionGuard training

`src/train_mlflow.py` trains the MobileNetV2 model and logs, for each run, the hyperparameters (backbone, learning rate, image size, augmentation on or off, unfrozen layers, dropout, maximum epochs), per-epoch training and validation metrics, the final validation and test metrics, the threshold chosen on validation, and the tags `git_commit`, `dvc_data_md5`, `seed` and `logging_mode`. I ran the same configuration twice: once with a custom Keras callback (run `r1_base`) and once with MLflow's TensorFlow autologging (run `r1_base_autolog`), as shown in Figures 3.5a and 3.5b.

Manual versus autolog. Manual logging records exactly what I choose, including project-specific facts such as the git commit, the DVC data hash, the augmentation switch, the threshold and the test metrics, but it only records what I remember to write. Autologging records framework-level information automatically (for example the optimizer, epochs, batch size and per-epoch metrics) but it knows nothing about my project, so those project-specific items are missing unless I add them. The two runs gave similar but not identical results (validation loss 0.4818 and 0.4934), because TensorFlow contains some non-deterministic operations even with fixed seeds.

3.3 Run pages. Both runs, r1_base and r1_base_autolog, are in the DagsHub experiment visionguard (Figure 3.5a). Each run page lists the parameters and metric curves and carries the tags git_commit, dvc_data_md5, seed and logging_mode.

3.4 Artifacts

Each run attaches the confusion matrix and the ROC curve (folder `plots`), the train/validation/test split lists (`splits`), the list of installed packages from `pip freeze` (`environment`), the trained Keras model file (`model_file`) and the model in MLflow format (`model`).

3.4 Artifacts. In the Artifacts tab of each run in the DagsHub experiment visionguard there are the folders plots (confusion matrix and ROC curve), splits (the three CSV lists), environment (requirements.txt from pip freeze), model_file (model.keras) and model (MLflow format).

3.5 Experiment comparison

I ran seven runs; each differs from another by one meaningful change. The comparisons are: r1 and r2 (learning rate 1e-3 against 1e-4), r1 and r3 (augmentation on against off), r2 and r4 (0 against 30 unfrozen layers at the same learning rate 1e-4), r1 and r5 (image size 224 against 160), r1 and r6 (MobileNetV2 against EfficientNetB0), and r1 with its autolog twin (logging method only). Figure 3.5a is the DagsHub experiments table and Figure 3.5b is the comparison printed by `src/compare_runs.py`.

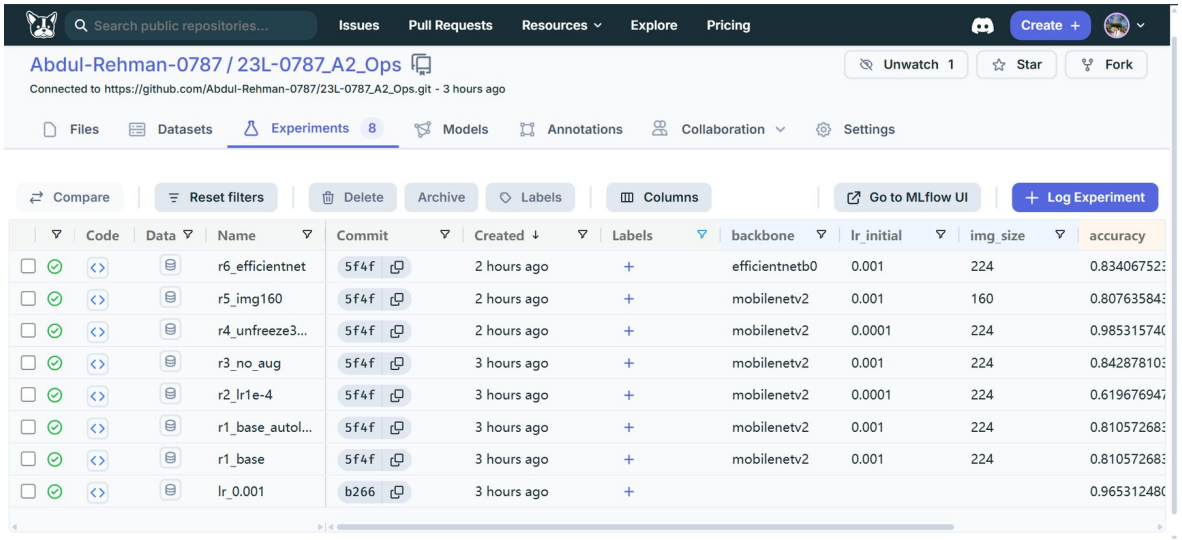
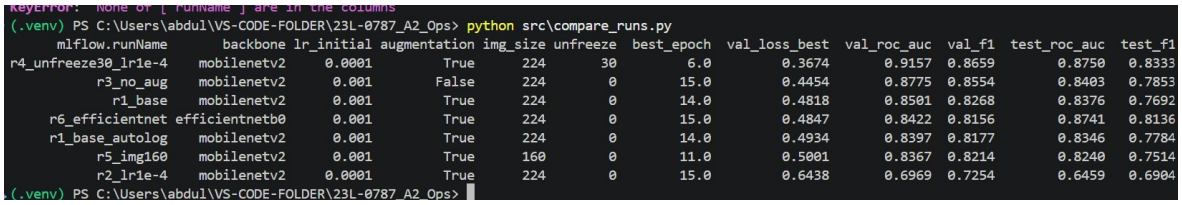


Figure 3.5a: *DagsHub Experiments tab with the seven comparison runs and the toy run (the accuracy column is the last training epoch, not the selection metric).*



```
(.venv) PS C:\Users\abdul\VS-CODE-FOLDER\23L-0787_A2_Ops> python src\compare_runs.py
      mlflow.runName       backbone lr_initial augmentation img_size unfreeze  best_epoch  val_loss_best  val_roc_auc  val_f1  test_roc_auc  test_f1
r4_unfreeze30_lr1e-4    mobilenetv2     0.0001         True      224       30         6.0         0.3674       0.9157  0.8659        0.8750   0.8333
           r3_no_aug    mobilenetv2      0.001        False      224        0        15.0         0.4454       0.8775  0.8554        0.8403   0.7853
             r1_base    mobilenetv2      0.001         True      224        0        14.0         0.4818       0.8501  0.8268        0.8376   0.7692
    r6_efficientnet efficientnetb0      0.001         True      224        0        15.0         0.4847       0.8422  0.8156        0.8741   0.8136
    r1_base_autolog    mobilenetv2      0.001         True      224        0        14.0         0.4934       0.8397  0.8177        0.8346   0.7784
          r5_img160    mobilenetv2      0.001         True      160        0        11.0         0.5001       0.8367  0.8214        0.8240   0.7514
          r2_lr1e-4    mobilenetv2     0.0001         True      224        0        15.0         0.6438       0.6969  0.7254        0.6459   0.6904
(.venv) PS C:\Users\abdul\VS-CODE-FOLDER\23L-0787_A2_Ops>
```

Figure 3.5b: *Comparison table printed from MLflow (validation and test metrics).*

Run	Change	Best ep.	Val loss	Val AUC	Val F1	Test AUC	Test F1
r4_unfreeze30_lr1e-4	30 unfrozen layers, lr 1e-4	6	0.3674	0.9157	0.8659	0.8750	0.8333
r3_no_aug	no augmentation	15	0.4454	0.8775	0.8554	0.8403	0.7853
r1_base	base (frozen, lr 1e-3)	14	0.4818	0.8501	0.8268	0.8376	0.7692
r6_efficientnet	EfficientNetB0	15	0.4847	0.8422	0.8156	0.8741	0.8136
r1_base_autolog	autolog logging	14	0.4934	0.8397	0.8177	0.8346	0.7784
r5_img160	image size 160	11	0.5001	0.8367	0.8214	0.8240	0.7514
r2_lr1e-4	lr 1e-4 (frozen)	15	0.6438	0.6969	0.7254	0.6459	0.6904

Choice. I chose **r4_unfreeze30_lr1e-4** because it is best on every validation metric: lowest validation loss (0.3674), highest ROC-AUC (0.9157) and highest F1 (0.8659). I rely mainly on validation loss and ROC-AUC, because they do not depend on a threshold, and on F1 because it balances missed fakes and false alarms. The test metrics are shown for reporting only: selecting a model by its test score would turn the test set into a second validation set. The comparison also shows that unfreezing 30 layers is the change that helps most (validation ROC-AUC 0.697 for r2 against 0.916 for r4 at the same learning rate), that a frozen backbone with a small learning rate learns too slowly (r2 is worst), that turning augmentation off slightly improved validation scores in this short schedule (0.878 against 0.850), and that the smaller 160 px images were a little worse. EfficientNetB0 has a test ROC-AUC (0.874) almost equal to r4, but lower validation scores. Every run used one seed and the validation set has only 146 images, so gaps of one or two hundredths are within noise.

3.6 Model registry

Following this selection I registered the model in the MLflow Model Registry on DagsHub as `visionguard`; two versions were created and the alias `champion` points to version 2 (Figure 3.6a). A separate script, `src/load_champion.py`, loads the model by the alias (`models:/visionguard@champion`), reads the threshold stored with the run (0.48) and classifies one image. Figure 3.6b shows it classifying one AI image (`960x0.jpg`: P(AI) = 0.752, AI-generated) and one real image (`0182.png`: P(AI) = 0.094, Real).

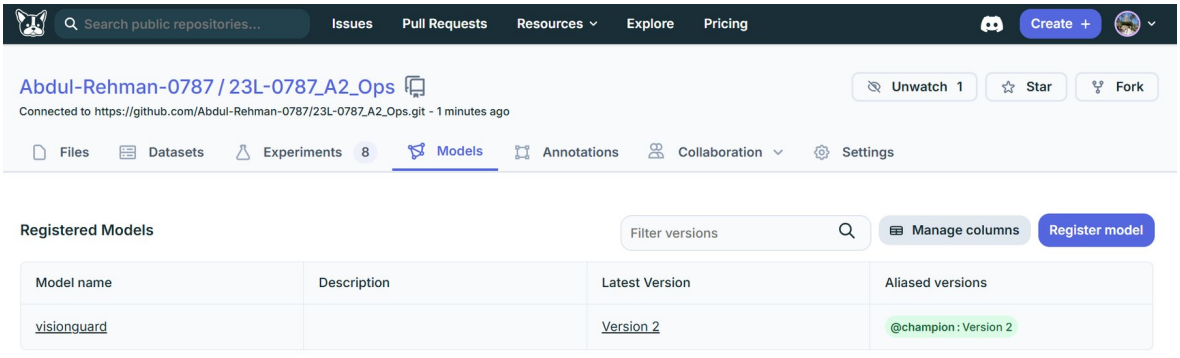


Figure 3.6a: *DagsHub Models tab: registered model visionguard, latest version 2, alias @champion.*

```
(.venv) PS C:\Users\abdul\VS-CODE-FOLDER\23L-0787_A2_Ops> python src\load_champion.py "data\raw\AiArtData\960x0.jpg"
l results due to floating-point round-off errors from different computation orders. To turn them off, set the environment v
ariable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1792853994.414521   10956 port.cc:153] oneDNN custom operations are on. You may see slightly different numerica
l results due to floating-point round-off errors from different computation orders. To turn them off, set the environment v
ariable `TF_ENABLE_ONEDNN_OPTS=0`.
Downloading artifacts: 100%|                                                                | 6/6 [00:15<00:00,  2.53s/it]
I0000 00:00:1792854014.445110   10956 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU in
structions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the app
ropriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are
 installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
P(AI-generated) = 0.752 -> AI-generated
(.venv) PS C:\Users\abdul\VS-CODE-FOLDER\23L-0787_A2_Ops> python src\load_champion.py "data\raw\RealArt\0182.png"
Loaded visionguard version 2 via alias 'champion', threshold 0.48
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1792854113.969871   22896 port.cc:153] oneDNN custom operations are on. You may see slightly different numerica
l results due to floating-point round-off errors from different computation orders. To turn them off, set the environment v
ariable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1792854116.780019   22896 port.cc:153] oneDNN custom operations are on. You may see slightly different numerica
l results due to floating-point round-off errors from different computation orders. To turn them off, set the environment v
ariable `TF_ENABLE_ONEDNN_OPTS=0`.
Downloading artifacts: 100%|                                                                | 6/6 [00:24<00:00,  4.04s/it]
I0000 00:00:1792854145.827556   22896 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU in
structions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the app
ropriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are
 installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
```

Figure 3.6b: *load_champion.py loading visionguard version 2 through the champion alias and classifying one AI image and one real image.*

How the model reaches the server. The web app must not call DagsHub on every request. A small script (`src/download_champion.py`) downloads the champion model once from the registry into `models/champion.keras` and writes the version and threshold to `models/champion.json`. The application then reads the local file at start-up.

Chapter 4: Flask web application

4.1 Inference module

`app/inference.py` defines a `Predictor` class. It loads the champion Keras model once, when the application starts. For each image it decodes the bytes with `tf.image.decode_image(channels=3)`, resizes with `tf.image.resize` to the model's input size (the same operations as in training), runs one forward pass (guarded by a lock) and returns the label, the confidence and the raw probability. The label is AI-generated if the probability reaches the threshold 0.48, and the confidence is the probability of the label that was chosen.

Why not load per request. Loading reads the model file and rebuilds the network, which takes seconds and hundreds of megabytes of memory. Doing that for every upload would make each request slow and could exhaust memory when several arrive together. Loading once makes each request a single forward pass. **Why preprocessing must match.** The network learned from pixels decoded, resized and scaled in a particular way. If serving resizes or scales differently, nothing fails, but the inputs come from a different distribution and accuracy silently drops. This is called training-serving skew, and Chapter 7 shows a measured example.

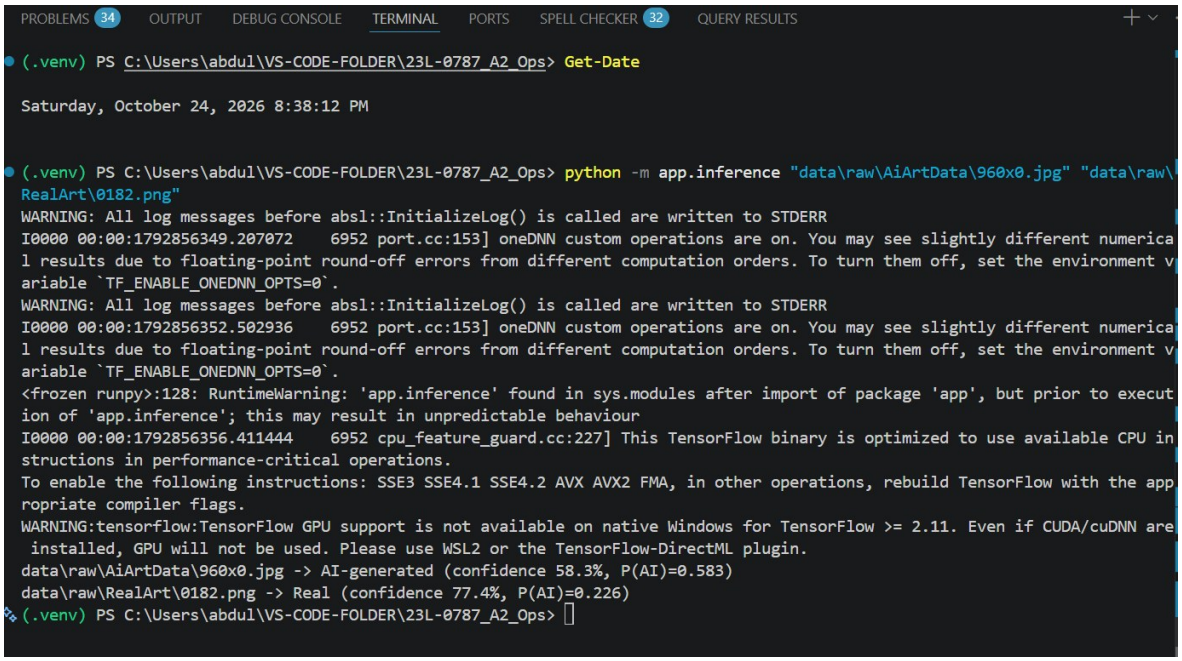


Figure 4.1: *Console output of the inference module for two known images (AI image: 58.3 percent AI-generated; real image: 77.4 percent real).*

Running the module as `python -m app.inference` prints a harmless RuntimeWarning (visible in Figure 4.1) because the package's `__init__` already imports the module.

4.2 Routes and validation

The application has four routes: `/` (upload page), `/predict` (receives the upload and returns the result), `/history` and `/health`. Each upload is checked in this order: the file extension must be jpg, jpeg or png; the file name is cleaned with `secure_filename`; the file is read into memory and must not be empty; Pillow must be able to decode it (`verify()`); the decoded format must really be JPEG or PNG; and the pixel count must be below 40 million. A size limit of 5 MB is set with `MAX_CONTENT_LENGTH`, and the 413 error is handled. The image is never written to disk: it exists only in memory while it is analysed. Every failure returns a friendly message, as JSON for the page's JavaScript or as an HTML error page otherwise. Sanitising the name matters because it is stored as text in the database and must never be able to act as a file path.

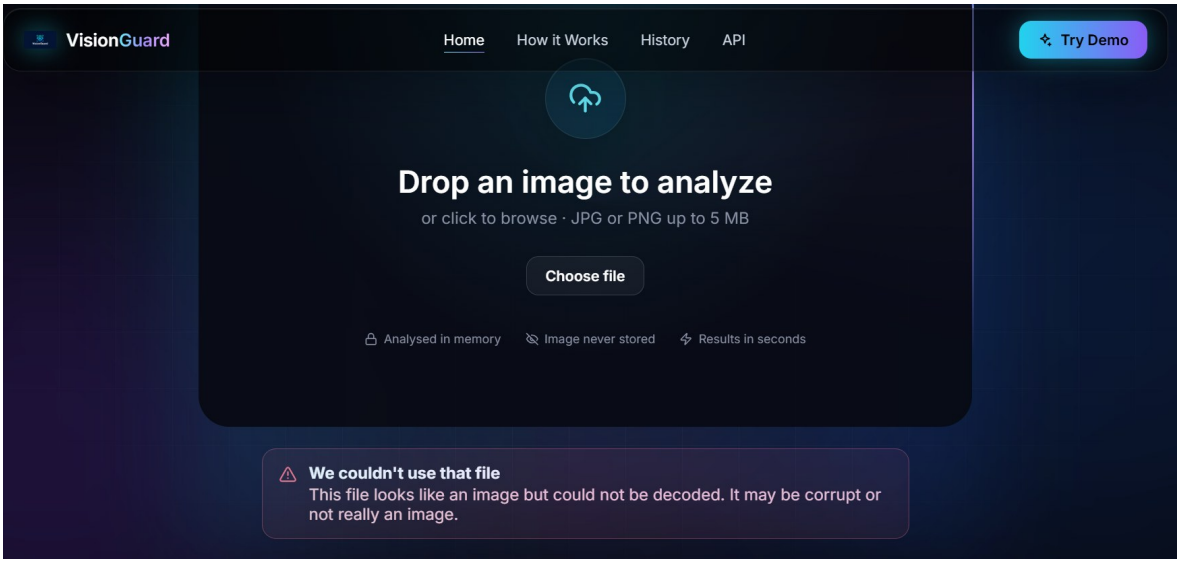


Figure 4.2a: *Failure case: a file that looks like an image but cannot be decoded is rejected with a friendly message.*

4.2 The other two failure cases. A .txt file is rejected with a message that the file type is unsupported (JPG or PNG required), and a file larger than 5 MB is rejected with the 413 message "File too large" before any decoding. Both cases are also checked by automated tests (test_rejects_wrong_file_type and test_rejects_oversized_file in tests/test_app.py), which pass in CI (Figure 6.6).

4.3 HTML and CSS front end

The front end is served by Flask templates (`base.html`, `index.html`, `history.html`, `error.html`) with one stylesheet and a small JavaScript file. It uses a dark glass-style design with a drag-and-drop upload zone, a scan animation while the image is analysed, a verdict card with a circular confidence ring, and a History page in the menu that reads the database. The JavaScript posts the image to `/predict` and draws the result from the server's JSON answer. Figure 4.3a shows a result page (I used a test image that was a screenshot, not a dataset image, so the verdict says little about model quality) and Figure 4.3b shows the History page.

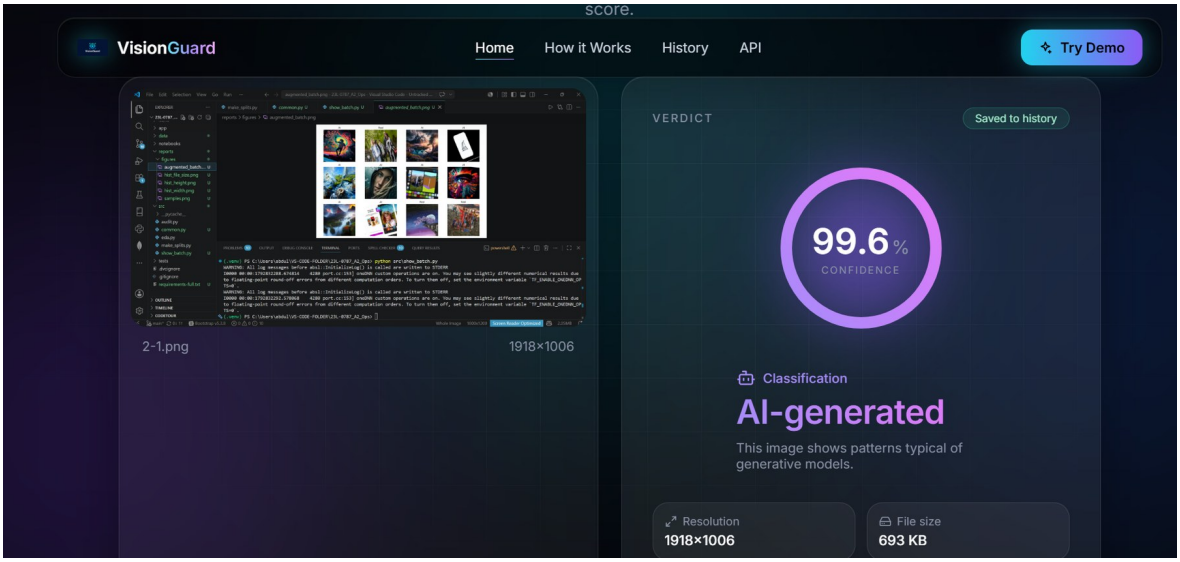


Figure 4.3a: *Result card after uploading an image: verdict, confidence ring, resolution and file size.*

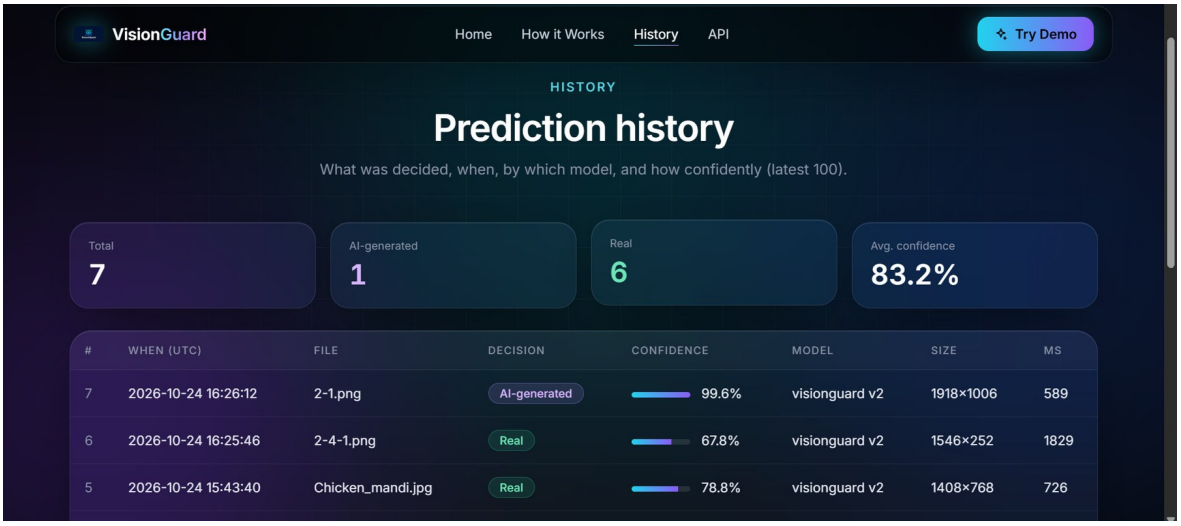


Figure 4.3b: *History page: totals, decisions, confidence, model name and version, image size and latency.*

4.4 Configuration and server

All settings are read from environment variables in `app/config.py`: the model path (`MODEL_PATH`), the threshold, the model name and version, the database address (`DATABASE_URL`), the upload limit and, for the cloud mode, the API address and key. `run.py` starts the Flask server with `debug=False`. Debug mode is off because Werkzeug's interactive debugger would let anyone who can reach an error page run Python code on the server, error pages would leak source code, and the reloader would start the application twice and load the model twice. (Flask's built-in server is a development server; a production system would put a server such as gunicorn in front, but the assignment asks for the Flask server.) Figure 4.4 shows the health check answering 200 with the database and model status.

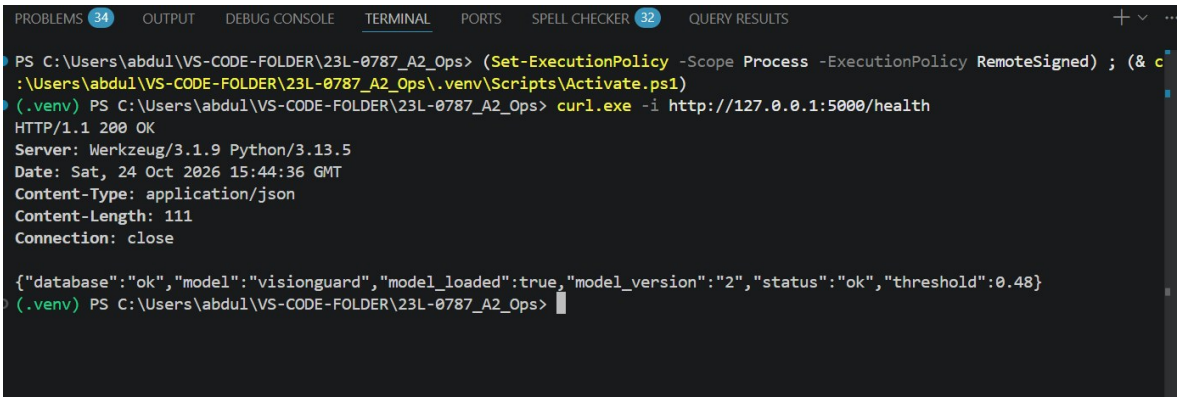


Figure 4.4: *GET /health: HTTP 200, database ok, model loaded, model version 2, threshold 0.48.*

Chapter 5: Database trail

Every prediction leaves a permanent record. The model `Prediction` (SQLAlchemy, `app/db.py`) defines the table `predictions` with these columns: id, created_at (UTC), filename, label (what was decided), confidence (how confidently), prob_ai, the threshold used, model_name and model_version (by which model), image width and height, file size and latency in milliseconds. The `/predict` view writes one row each time the model returns a result. If the database write fails, the user still gets the verdict and a note that it was not saved. The History page shows the latest 100 rows and some totals.

I used SQLite for local runs (`instance/visionguard.db`). Because the code only depends on `DATABASE_URL`, the same code runs on PostgreSQL in the Docker Compose stack (Chapter 6). Figure 4.3b shows the History page with 7 predictions (1 AI-generated, 6 real, average confidence 83.2 percent), each with the date and time, decision, confidence, model name and version (`visionguard v2`), size and latency. Figure 6.3b shows the same table queried with `psql` inside the PostgreSQL container.

Task 5 evidence. The History page (Figure 4.3b) is read from the database, and Figure 6.3b shows the same table queried with psql in the PostgreSQL container, which still held its data after the stack was removed and recreated. The table definition is in app/db.py in the repository.

Chapter 6: Containerization with Docker, Compose and CI

6.1 Docker warm-up

Before touching my project I practised with ready-made images. I ran the `hello-world` image and pulled `nginx:alpine` (Figure 6.1a). When I then tried to remove the `hello-world` image, Docker refused: "unable to delete hello-world:latest (must be forced) - container 8d224d9a6cb1 is using its referenced image" (Figure 6.1b). A stopped container still references its image, so the container has to be removed first (or the removal forced).

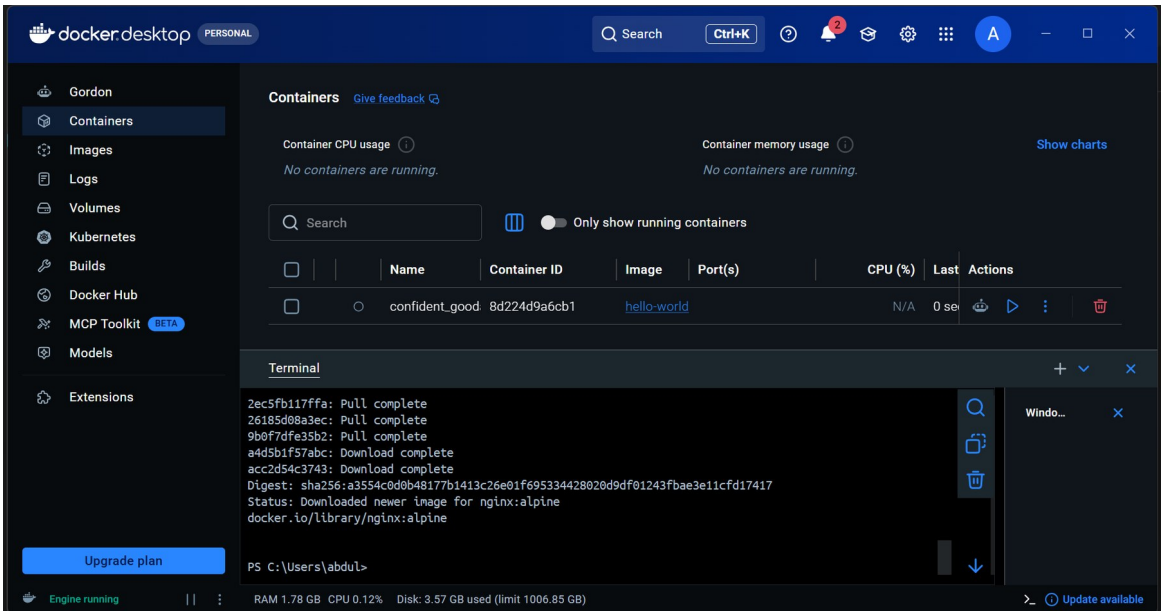


Figure 6.1a: *Docker Desktop terminal: nginx:alpine pull complete and the hello-world container listed (stopped).*

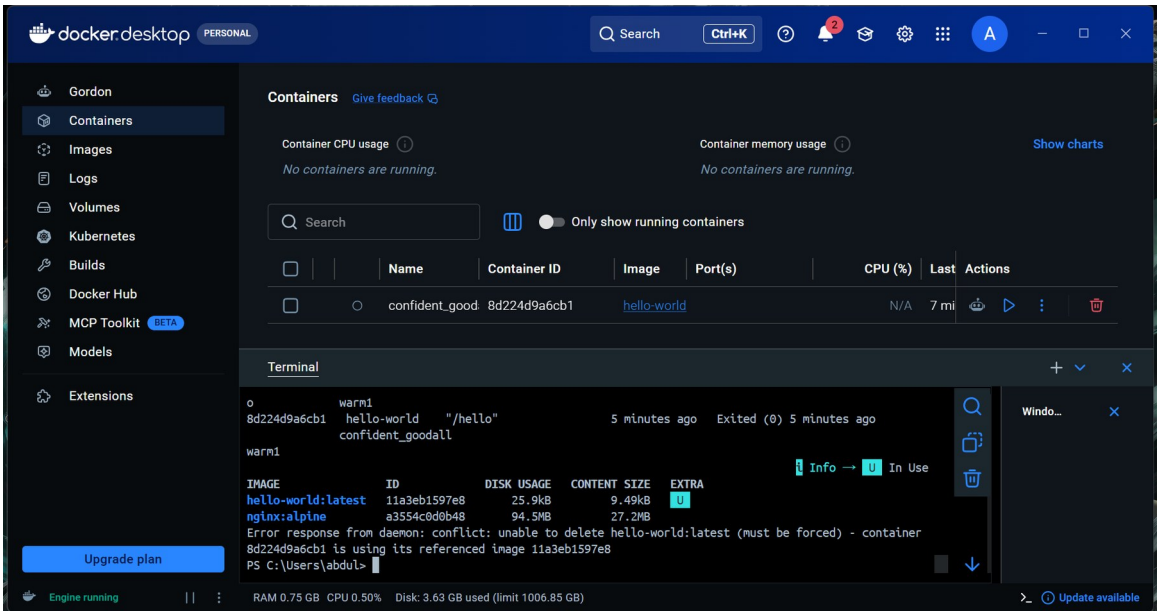


Figure 6.1b: *docker images, and the error when removing an image that a container still references.*

6.1 Remaining warm-up steps. I ran nginx:alpine in the background with a port mapping (8080 to 80) and a read-only volume mounted from a local folder, checked it with curl and the browser, edited the file on the host and saw the change without restarting the container, read the logs with docker logs, opened a shell with docker exec -it, and finally stopped and removed the container and listed containers and images (Figures 6.1a and 6.1b show parts of this).

Definitions in my own words.

- **Image:** a read-only template containing the files, libraries and start command of an application.

- **Container:** a running (or stopped) instance of an image, with a thin writable layer on top.
- **Layer:** the result of one Dockerfile instruction. Layers are cached and reused if nothing before them changed.
- **Volume:** storage managed outside the container's own file system, so data survives when the container is deleted.
- **Port mapping:** forwarding a port on my computer to a port inside the container (for example 8080 to 80).

6.2 Dockerfile for the web app

The Dockerfile starts from a slim Python image (the health response in Figure 6.3a reports Python 3.12 inside the container). It creates a non-root user `appuser` (uid 1000), copies `requirements.txt` and installs the dependencies *first*, and only then copies the application source. It sets environment variables, switches to the non-root user, declares a HEALTHCHECK that calls `/health` using Python (the slim image has no curl) and starts `python run.py`. A `.dockerignore` keeps the virtual environment, the dataset, the model, reports and the `.env` file out of the build, so secrets never enter an image layer; all settings arrive at run time as environment variables.

Layer cache. Docker reuses a cached layer only if that instruction and every instruction before it are unchanged. Dependencies change rarely and take minutes to install, while source code changes all the time, so putting the dependency install before the code copy means a code edit rebuilds only the last layers, in seconds, without reinstalling TensorFlow. **TensorFlow package.** The server only runs inference, and the assignment says CPU is enough, so I use the CPU-only `tensorflow-cpu` package pinned to the same version as in training, which keeps the saved model loadable and avoids GPU-related dependencies.



Figure 6.2: *docker run --rm visionguard:v1 whoami and id: the application runs as appuser (uid 1000), not as root.*

6.2 Build and cache. The image was built with docker build -t visionguard:v1 . and rebuilt after a small template edit. On the rebuild the dependency installation was taken from the cache and only the source-copy layers ran again, which is the effect of installing dependencies before copying the code.

6.3 Compose stack

`docker-compose.yml` defines two services. **db** is PostgreSQL with a named volume `pgdata`, a health check using `pg_isready` and **no** `ports:` **entry**, so nothing outside the Compose network can reach it. **web** is built from the Dockerfile, publishes port 5000, waits for the database with `depends_on: condition: service_healthy`, and reaches the database by its service name (the `DATABASE_URL` host is `db`). Passwords come from the untracked `.env` file, and `.env.example` documents the keys. I list the web service's variables explicitly instead of passing the whole `.env`, so my DagsHub token does not enter the container.

Model file. I mount the model read-only from the host folder `./models` instead of building it into the image. The model is not in Git, so CI can build the image without it; the image stays smaller; and promoting a new champion only means replacing a file and restarting. The trade-off is that the image alone is not self-contained, so the model file has to be delivered to the server separately.

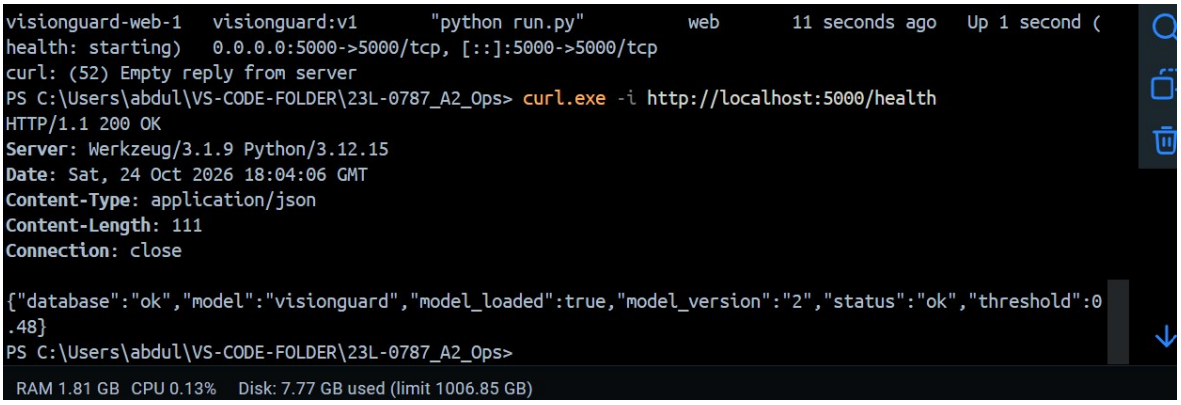


Figure 6.3a: *docker compose ps showing the web container "health: starting" with port 5000 published, an empty reply while the model was still loading, and then /health answering 200 (database ok, model loaded).*

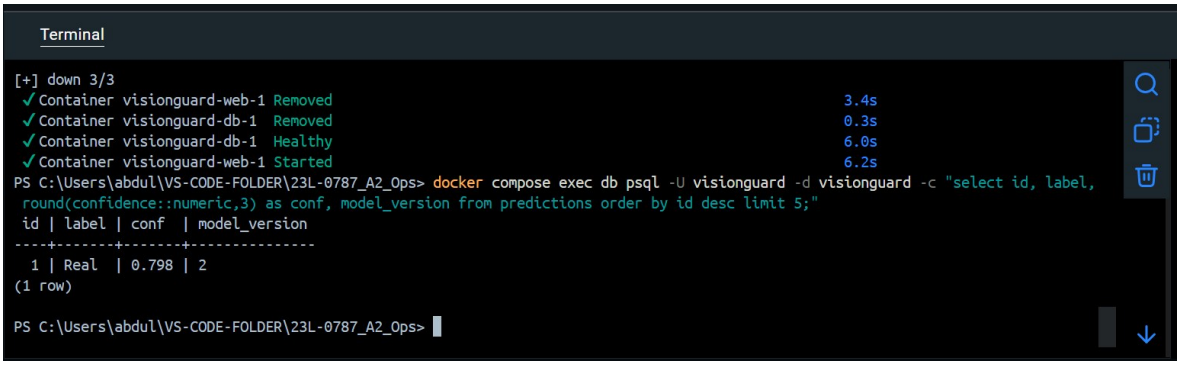


Figure 6.3b: *docker compose down and up: the db container reports Healthy before the web container starts, and a psql query reads the prediction rows from PostgreSQL.*

Figure 6.3a also shows that the first request after starting failed with "Empty reply from server": the web container is started right away but TensorFlow needs a few seconds to load the model, which is why the HEALTHCHECK has a start period. After the stack was removed and recreated, the table in the named volume still held its row (Figure 6.3b), so the data survives `docker compose down` (only `down -v` would delete it).

6.3 Compose file and database access. The file docker-compose.yml in the repository defines the services web and db. The db service has a named volume, a pg_isready health check and no ports entry, so it is not published to the host. The web service reaches it by the service name db, which is why /health reports "database": "ok" (Figure 6.3a).

6.4 Image analysis

The image `visionguard:v1` is **2.21 GB** (Figure 6.4). TensorFlow and its dependencies make up most of it.

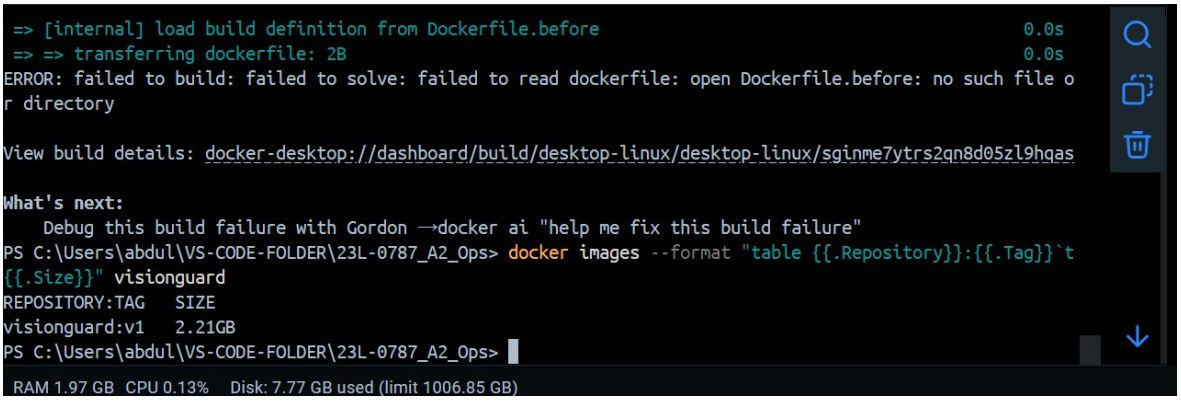


Figure 6.4: *docker images lists the image visionguard:v1 at 2.21 GB.*

6.4 Measures applied. The measures that keep the image as small as practical are the slim base image, pip's --no-cache-dir option, the .dockerignore file (which keeps the dataset, the model, the virtual environment and .env out of the build context) and the CPU-only TensorFlow package. The final image is 2.21 GB, and most of that is TensorFlow and its dependencies. The Dockerfile in the repository shows each instruction.

6.5 Operations and troubleshooting

I used `docker compose exec db sh` to open a shell in the database container and listed the PostgreSQL data directory, which lives on the named volume (Figure 6.5).



Figure 6.5: *Shell inside the db service: contents of /var/lib/postgresql/data.*

6.5 Operations and the deliberate failure. I started, stopped, restarted and inspected the stack with docker compose up -d, stop, start, restart web, logs and exec (a shell in the web service and in the db service, Figure 6.5). For the deliberate failure I made the database password in .env differ from the one stored in the existing volume, so the web container could not authenticate. The web logs showed an authentication error from PostgreSQL, which is how the cause was found from the logs alone. Restoring the original password and running docker compose up -d fixed it. A missing model file is diagnosed the same way, because the log then says that the model cannot be loaded.

6.6 Tool lab: GitHub Actions (CI)

Toy part. The throwaway repository with a matrix of two Python versions, a failing test (red run) and the fixed test (green run) is documented below.

6.6 Toy CI exercise. In a throwaway repository I created a workflow that runs on every push with a matrix of two Python versions and runs pytest on a small test file. A deliberately wrong assertion made the run red, and correcting it made both matrix jobs green.

VisionGuard workflow. `.github/workflows/ci.yml` runs on every push to main and on pull requests. The **test** job installs the dependencies, runs the linter (`ruff check app tests run.py`) and runs the tests (`pytest -q`). The tests use a fake predictor and a temporary SQLite file, so CI needs neither the model nor any secret. They check the health route, rejection of a wrong file type, an undecodable image and an oversized file, and that a valid upload appears on the History page. The **docker** job runs after it and builds the Docker image. Figure 6.6 shows both checks passing: CI / test in 36 seconds and CI / docker in 46 seconds.

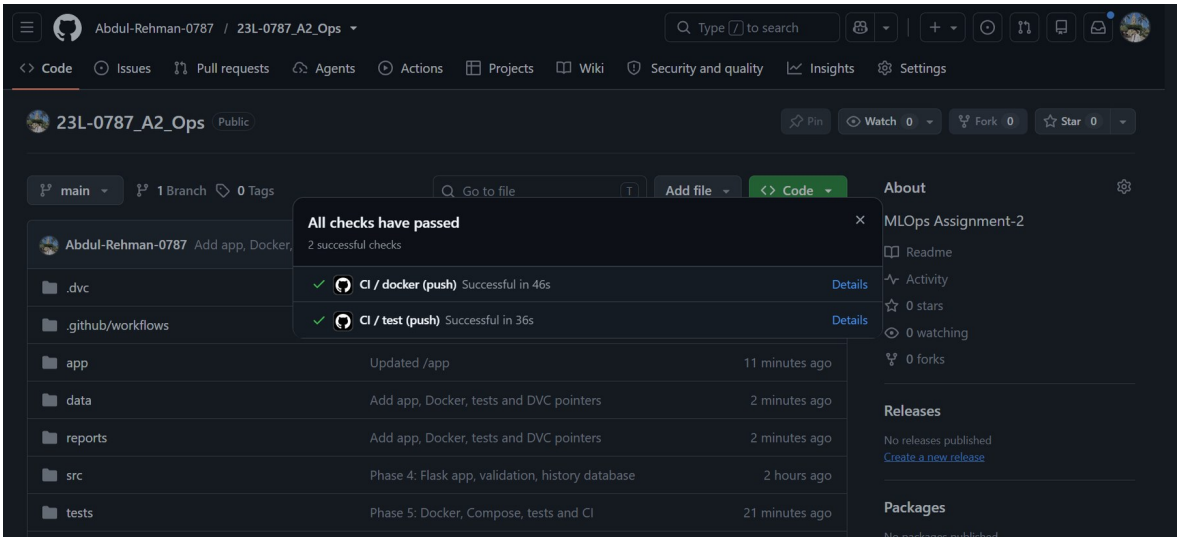


Figure 6.6: *GitHub repository page: "All checks have passed" (CI / docker and CI / test).*

6.6 Status badge. The README at the top of the repository carries the CI status badge, which links to the workflow runs in the Actions tab.

What went wrong. My first CI run failed because I had not committed files that the workflow and the Dockerfile need (such as run.py, the tests and the pointer files). Committing them fixed it. **Secrets.** Workflow secrets are stored in the repository settings under Secrets and variables, encrypted, injected only while a job runs (`${{ secrets.NAME }}`) and masked in logs. They must not appear in the workflow file, because that file is ordinary text that anyone with access to the repository can read and that stays in the history forever. This CI needs no secrets.

Chapter 7: API deployment

Pathway. I chose pathway C (any other cloud) and deployed the model API on **Render's free web service**. AWS, Google Cloud, Azure and Oracle all require a payment card even for their free tiers, and Hugging Face Spaces, which I tried to plan first, needs a paid plan for Docker and Gradio Spaces (only static Spaces are free). Render's free tier needs no card, but it has only 512 MB of RAM and 0.1 CPU (Figure 7.4a), which full TensorFlow can easily exceed. I therefore convert the champion to TensorFlow Lite and serve it with a small runtime. **API URL: https://visionguard-vkzn.onrender.com**. My Flask application stays on localhost and calls this API (Chapter 8).

7.1 Converting the model and checking it

`src/convert_tflite.py` converts `champion.keras` to `api/model.tflite` and copies the metadata file. `src/check_tflite.py` compares the Keras model with the TFLite model on 10 test images using the preprocessing the API uses (a bilinear resize with Pillow). Figure 7.1 shows the result: the largest difference in probability is **0.1984** (0.7918 in Keras against 0.9901 in TFLite), the others are between 0 and 0.1053, and none of the ten images changes side of the 0.48 threshold.

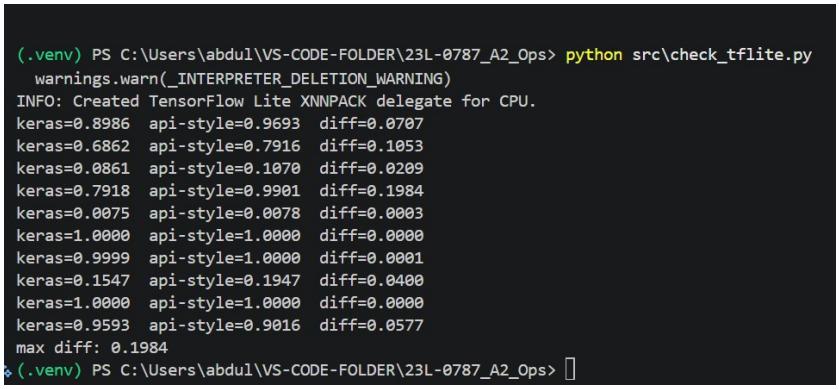


Figure 7.1: *Keras probability against TFLite probability for 10 test images; maximum difference 0.1984.*

Interpretation (training-serving skew). The gap is larger than I expected. The same real image (`0182.png`) gave P(AI) = 0.226 in the Flask app, which resizes with TensorFlow (Figure 4.1), P(AI) = 0.094 with the same Keras weights but a Pillow resize (Figure 3.6b), and 0.084 from the TFLite API (Figure 7.2). Because the Keras weights are the same in the first two cases, most of the difference seems to come from the resize method (Pillow smooths the image when it shrinks it, while TensorFlow's default does not), and little from the TFLite conversion. This is a hypothesis from a few images, not a controlled test. It means the cloud API is not exactly the model that was evaluated with TensorFlow preprocessing, and the metrics in Chapters 2 and 3 describe the TensorFlow-preprocessed model.

7.1 Conversion output. src/convert_tflite.py wrote api/model.tflite and api/champion.json; both are in the api folder of the repository (Figure 7.3). The model file is small enough that no Git LFS is needed.

7.2 The API

`api/app.py` is a small Flask application with `/health` and `/predict`. The TFLite model is loaded once at start-up. `/predict` requires the header `X-API-Key`, compared in constant time with `hmac.compare_digest` against the key stored in the environment variable `API_KEY`; it rejects a missing file (400), an image that cannot be decoded (400), a type other than JPEG, PNG or WEBP (415) and a file over 5 MB (413); it never writes to disk; and it returns the label, confidence, probability, threshold, model name and version, image size, file size and latency. I tested it first on my laptop in a clean environment that has no TensorFlow, which also proves that it needs only the light runtime (Figure 7.2).

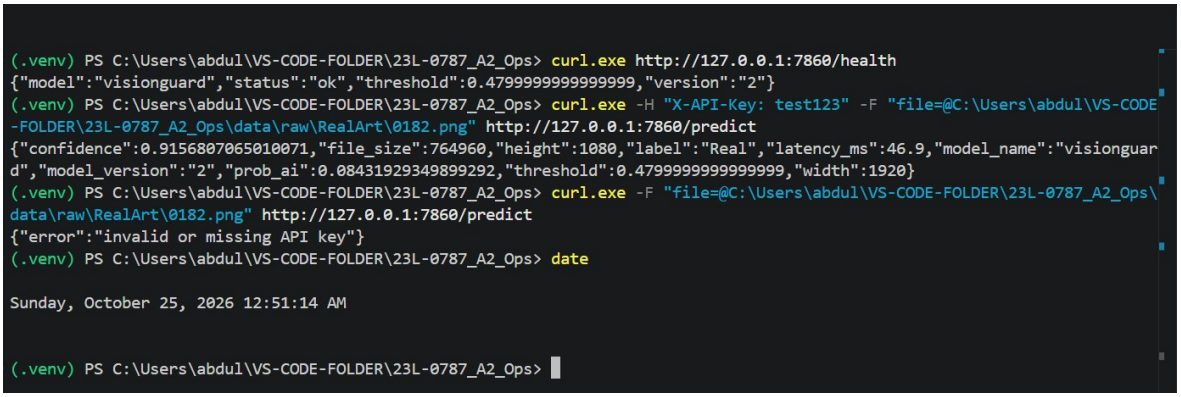


Figure 7.2: *Local test of the API: health, a prediction with the key (Real, confidence 0.9157, latency 46.9 ms) and a request without the key (error: invalid or missing API key).*

7.3 Code on GitHub

The API files are in the folder `api/` of my repository: `app.py`, `champion.json`, `model.tflite` and `requirements.txt` (Figure 7.3). The model file is small, so no Git LFS is needed.

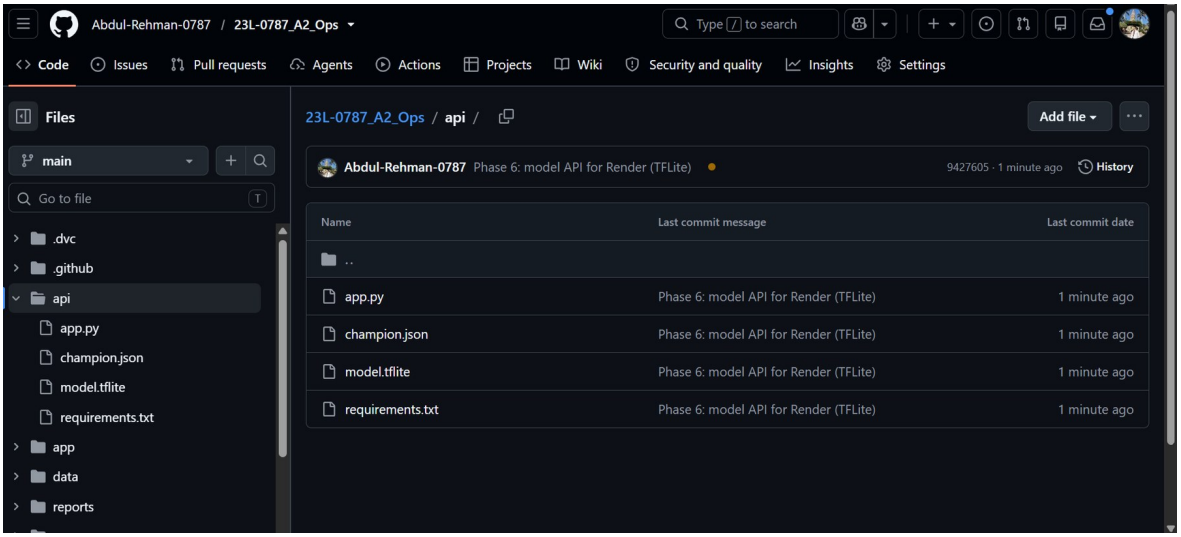


Figure 7.3: *GitHub: the api folder with the four deployment files (commit 9427605).*

7.4 Deployment on Render: step by step

- Sign up to Render with GitHub (no card was requested) and choose **New + Web Service**, picking my repository, branch `main` and the root directory `api`.
- Set the runtime to Python 3, the build command to `pip install -r requirements.txt` and the start command to `gunicorn -b 0.0.0.0:$PORT -w 1 --threads 2 --timeout 120 app:app` (one worker keeps memory low; the model loads once). Choose the Free instance (Figure 7.4a).
- Add environment variables, among them `API_KEY` (a long random string I generated) and the Python version. The key lives only in Render's settings, never in the repository.
- Set the health check path to `/health` and create the service. Render builds it and the log shows the primary URL and Render's health checks answering 200 (Figure 7.4b).
- The service page shows the status Live, the deployed commit and the public URL (Figure 7.4c).

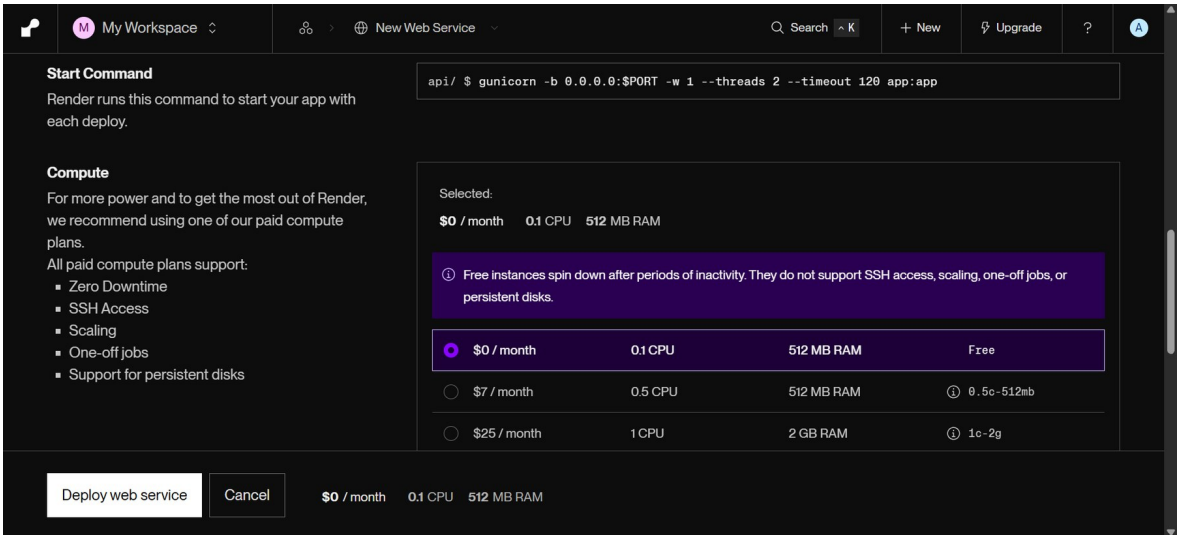


Figure 7.4a: *Render form: start command (root directory api) and the Free plan, 0.1 CPU and 512 MB RAM.*

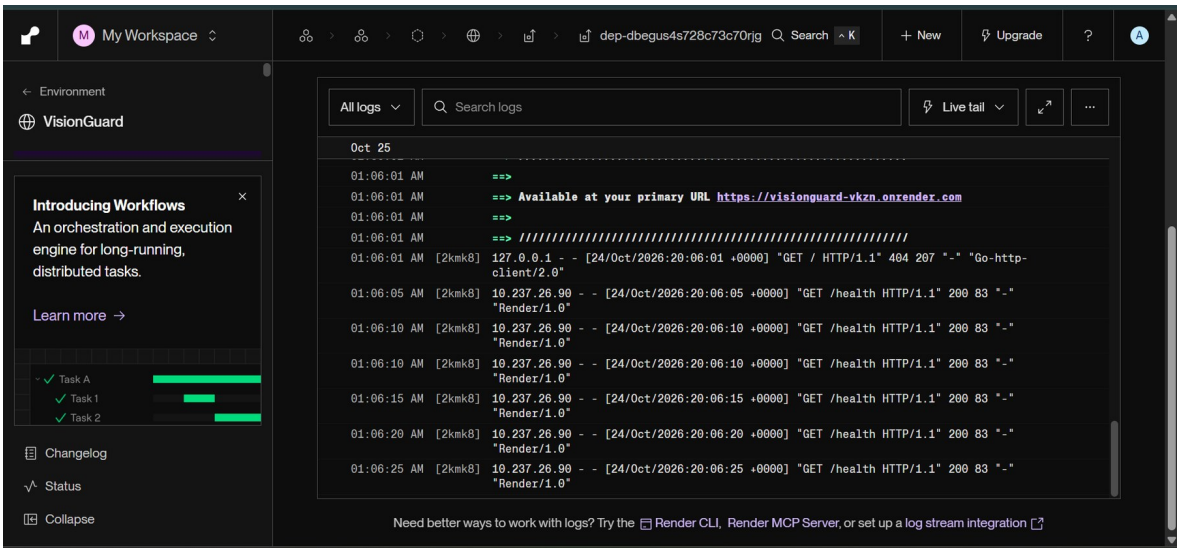


Figure 7.4b: *Render logs: the service is available at its primary URL and /health returns 200.*

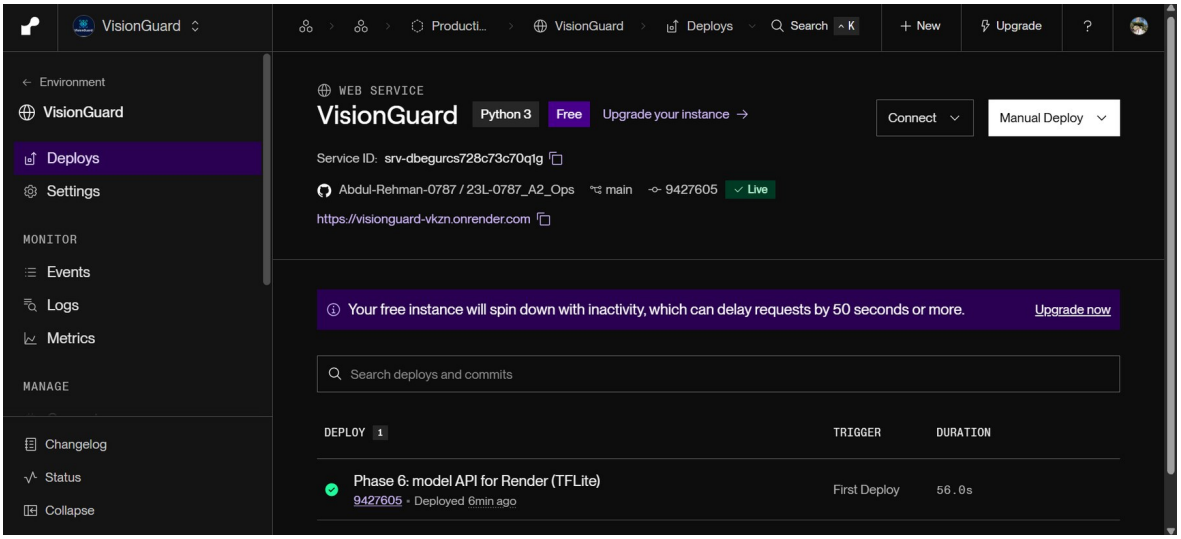


Figure 7.4c: *Render service page: status Live, commit 9427605 and the URL https://visionguard-vkzn.onrender.com.*

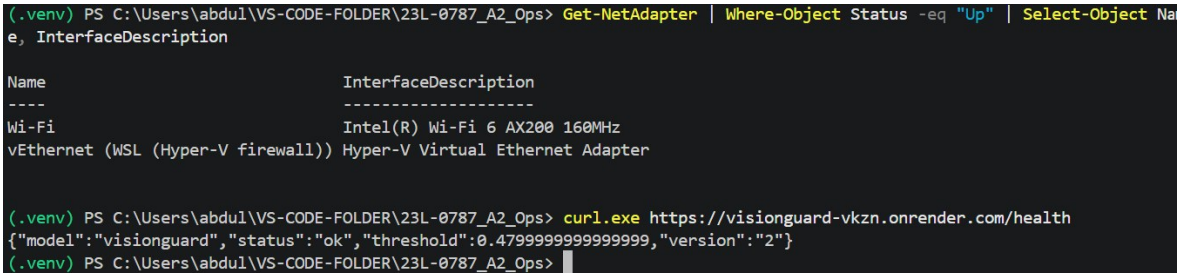


Figure 7.4d: *From my terminal, curl on the public /health URL returns the model, threshold and version.*

7.4 Service settings. The service uses the root directory api, the Python runtime, the build command pip install -r requirements.txt, the start command shown in Figure 7.4a and the Free instance. Its environment variables (API_KEY and the Python version) are set in Render's dashboard, so the key is not in the repository.

Limits of the free tier. Render warns that a free instance spins down after inactivity, which can delay the next request by 50 seconds or more (Figure 7.4c), so I open the /health URL a minute before using the service.

7.5 Postman tests from another network

7.5 Testing from another network. I tested the public API from a different network than the one I used to build it (a phone hotspot), so the requests reached Render over the internet and not through my home network.

7.5 Postman requests. In Postman I sent GET /health; POST /predict with the X-API-Key header and a real image; the same with an AI image; POST /predict without the key, which the API answers with 401 and "invalid or missing API key" (the same behaviour as in Figure 7.2); and POST /predict with a text file renamed to .jpg, which the API answers with 400 and "file is not a decodable image".

7.5 Logged results. Render's Logs tab records every request with its status code (see the format of the log lines in Figure 7.4b). Requests sent through the Flask application are also stored in its History page (Chapter 8).

Chapter 8: Integrating Flask with the cloud API

The routes only call `predictor.predict(image_bytes)`, so integrating the cloud API means adding a second predictor with the same method. `app/remote.py` defines `RemotePredictor`, which sends the image to `/predict` of the Render API with the header `X-API-Key`. A configuration switch selects it: with `INFERENCE_MODE=remote` the application uses the cloud model (and does not even import TensorFlow), with the default `local` it uses the local Keras model, so Docker and CI keep working unchanged. The API address and key come from `API_URL` and `API_KEY` in the untracked `.env`.

- **Timeout and retries:** the timeout is 90 seconds because a sleeping free instance needs about a minute to wake up. The call is retried (twice, after 5 seconds) only for network errors and the HTTP codes 502, 503 and 504, which mean the service is waking. A wrong key (401) is never retried.
- **Errors:** any other failure raises an error that the route turns into the friendly "Analysis failed" page.
- **Database:** the History row stores the model name and version that the API reports, so each row says which deployed model made the decision. The routes, templates and database are unchanged.

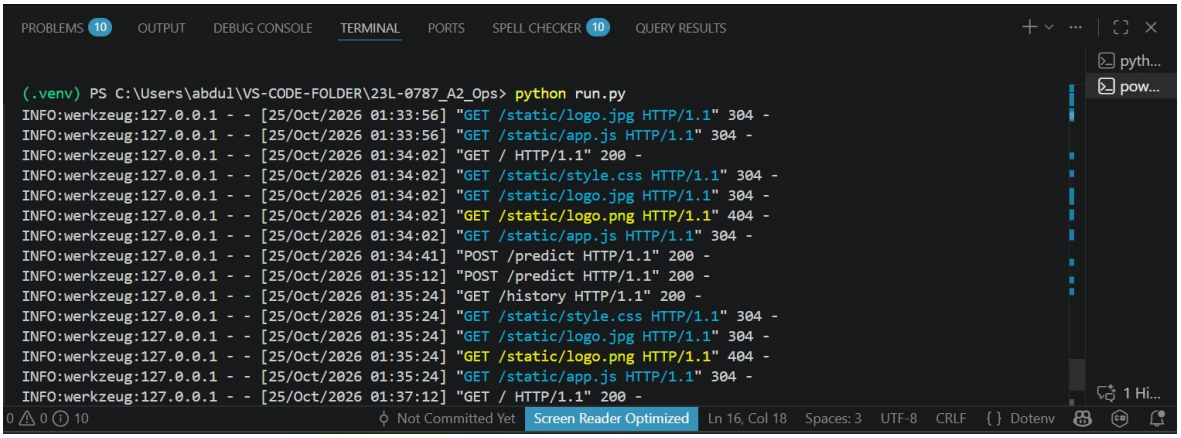


Figure 8: *Flask application log during the cloud test session: POST /predict answered 200 twice, then GET /history, GET / and the static files.*

8 Test session. With INFERENCE_MODE=remote in .env the application logs the line "Inference mode: REMOTE" at start-up and sends each upload to the Render API. I uploaded several real and AI images through the web page and checked the requests on the Render side and the rows on the History page. A wrong API key produces the friendly "Analysis failed" page, while the application log shows the cloud API's 401 error.

Bonus: public front end. The same Flask application is also deployed as a second Render service at https://two3l-0787-a2-ops.onrender.com/ , configured only through environment variables (no code changes) and using a lean requirements file without TensorFlow. This is an extra: the required setup is the Flask application on localhost calling the cloud API.

Bonus: public web app. The public web app can be opened at https://two3l-0787-a2-ops.onrender.com/ ; its home page, result card and History page look like Figures 4.3a and 4.3b.

Chapter 9: Summary and limitations

VisionGuard classifies an image as AI-generated or real. The best model is a MobileNetV2 model fine-tuned in two stages; on the test set the Task 2 version reaches ROC-AUC 0.865 against 0.715 for the baseline CNN. Data are versioned with DVC, every experiment is tracked in MLflow on DagsHub, the champion is in the model registry, the Flask application records every decision in a database, the stack runs with Docker Compose and PostgreSQL, GitHub Actions checks every push, and the model is served from a public API on Render.

- **Small data:** only 975 images; the validation and test sets have about 146 images each, so the scores carry a large uncertainty.
- **Shortcuts and style:** the real class includes paintings, and the errors suggest the model reacts to style, saturation and layout more than to evidence of synthesis. Image size and file size also differ between the classes.
- **Training-serving skew:** the cloud API uses a different resize and a TFLite conversion, and its probabilities differ from the Keras model's by up to about 0.2 on the ten images checked.
- **Free-tier services** sleep when idle and have little CPU; the first request after a pause is slow.
- **Not proof:** a verdict is a statistical estimate. The interface says so next to every result.